Local Coding Barriers and the Limits of Tape Geometry for Matrix Transposition

Hans Brende GPT-6 Astra (OpenAI)

October 2026

Abstract

We prove six families of results for finite coding models and for matrix transposition on deterministic multitape Turing machines, using the native model of *Matrix Transposition and the Tape Coding Conjecture*. For local coding, we obtain adaptive original-row separator bounds with optimal universal deficit and helper coefficients over finite instances; near-full broadcast bounds for all-input correctness after bijective nonlinear row recoding, when fixed coordinate-XOR packets follow a bounded-degree row graph; and a $b\lfloor d^2/4 \rfloor$ lower bound on helper entropy plus input entropy deficit for a specified two-stage architecture. Here d is the matrix dimension and b the entry width. For head motion, we bound time outside locally affine regions and exclude specified scan palettes and regular contracting recursions. We also construct, for every native machine, a fixed wrapper with additive $O(d^2 b)$ overhead on canonical inputs whose complete runs satisfy the original zero-slack cut inequality. Under a linear-time bound, its normalized head limits satisfy the full M^* conditions. Preparation, motion, cleanup and handoff are all charged. These geometric conditions can therefore be supplied independently of the computation that follows the prefix. All six families are formalized in Lean. Unrestricted linear-time transposition and integer multiplication remain unresolved here.

Contributions. Hans Brende directed the research, supplied sources, and reviewed the scope and interpretation of results. GPT-6 Astra (OpenAI) developed the arguments and Lean proofs, performed computational and source checks, and prepared the manuscript under his direction.

Contents

1 Introduction **3**
1.1 Proofs and their interpretation 4

2 Common notation and the native target **4**

3 Related work and attribution **5**
3.1 The shared tape model and the scope of the question 5
3.2 Local coding, nonlinear recoding, and helper information 6
3.3 Affine geometry, expanders, and paid computation 6
3.4 The transpose prediction conjecture is already false 7
3.5 Formal dependencies and novelty boundaries 7

4 Local coding obstructions **8**
4.1 Adaptive queries of original rows 8
4.1.1 A treewidth consequence 11

4.2 Bijective row recoding and fixed XOR packets . . . 12
4.3 Two nonlinear ordered stages and a global helper . . . 15

5 Affine motion and exceptional physical time 18
5.1 The crossing convention and the affine-time bound . . . 18
5.2 Native normalization and correct-input quantifiers . . . 20

6 Concrete palettes and contracting recursion 22
6.1 The declared palettes . . . 22
6.2 Paid contracting stage clocks . . . 26
6.3 Actual-stage regularity and the exceptional-time toll . . . 27

7 Fully paid padding and the limits of the motion test 29
7.1 The exact geometric contract . . . 29
7.2 The wrapper theorem . . . 30
7.3 The analytic interface and its attribution . . . 32
7.4 What the construction establishes . . . 32

8 What remains to decide 33

A Building and checking the artifacts 34
A.1 Shared sources and trust boundary . . . 34
A.2 Reproduction . . . 34
A.3 What the verification establishes . . . 35

B Declaration and dependency index 35

1 Introduction

An input in row-major order already contains every bit of its transpose. The difficulty on a tape machine is to deliver those bits in column-major order while paying for every movement and operation. Routing follows the bits through their successive locations. Coding instead allows a cell or message to depend on several input bits, with the desired bits recovered later. A routing lower bound therefore need not be a computational lower bound.

Brende and Claude Opus 5.5 [9] formulate the Tape Coding Conjecture and give sufficient conditions for excluding linear-time transposition. They also prove head-pass lower bounds and a macroscopic isoperimetric condition, Theorem M^*, for the work-head trajectories of fast transposers. Their construction discussion separates complicated head motion from the unresolved coding computation. This paper develops both sides of that separation.

Our results fall into two connected groups. The first concerns abstract local information access. Adaptive nonlinear broadcasts confined to original rows obey separator bounds; bijective nonlinear row recodings still require near-full broadcasts for bounded-degree fixed coordinate XORs under all-input correctness; and a particular two-stage architecture requires substantial global helper information. The second concerns actual tape execution. Theorem M^* imposes a positive lower bound on total time outside locally affine regions of the limiting head paths. This excludes specified operations and regular recursions. Nevertheless, a fully paid prefix supplies the original cut condition for any payload computation and the limiting-motion conditions for any linear-time payload.

What the padding construction shows. As a corollary of Theorem 5.2 applied to the M^* limits of Theorem 7.1, the padding's motion lies outside the excluded class. The padding theorem does not refute the Tape Coding Conjecture, whose demand and causal-code requirements are additional conditions. It establishes that the particular geometric inequalities under consideration can be satisfied without performing the transpose. To obtain a transposition lower bound, these inequalities must be combined with an additional restriction, such as one tracking the information required at the outputs. For a construction, satisfying them leaves encoding and decoding to be implemented.

Six families, with different scopes. The finite coding theorems permit general entry width b. Their hypotheses describe abstract broadcasts or row transformations; they are not automatic properties of arbitrary tape computations. The native asymptotic theorems use the canonical logarithmic width and the original finite-state machine model. The motion results and their applications allow negligible entropy deficit and, where stated, negligible size-dependent control information. The XOR theorems, in contrast, require all-input correctness. Each change of model is made explicit at the relevant statement.

Relation to the earlier paper. The local coding results turn parts of the earlier construction programme into proved restrictions; they are not improvements to an existing separator or two-stage theorem there. The affine results strengthen consequences of Theorem M^*. The paid construction gives a data-free version of the earlier expanding-motion proposal: it generates and executes the motion in linear time, before the payload begins. Coupling that motion to coded data at each recursive level remains open. The scan and recursion results give applications in the actual machine model. None improves an unrestricted transposition time bound.

1.1 Proofs and their interpretation

The proofs of our new results are the Lean proofs. Every numbered formal statement names its declarations and source modules. The Lean statements are authoritative; the English statements are consequences of them, with any specialization stated explicitly. Each *Outline of the formal proof* explains the checked argument for human readers. The declaration index in Appendix B collects these links.

Formal verification establishes propositions about the stated definitions; model applicability and novelty require separate arguments. Our review checks the English hypotheses, quantifiers, conclusions and proof outlines against the formal source. Imported theorems are proved dependencies. The original paper's four explicitly unformalized background propositions retain that status when cited here.

The machine definitions and original theorems are imported directly from the sibling project. The analytic chain additionally uses an unmodified pinned OpenAI formal source, as specified in Appendix A. The established expander mathematics and imported projection theorem are credited in Section 3, together with the literature comparisons for our contributions.

2 Common notation and the native target

Matrices and information. The dimension is d, the number of bits in an entry is b, and $P = d^2 b$. An input is a bit matrix $x = (x_{ijt})$ with $i, j < d$ and $t < b$. Its row-major encoding writes bit (i, j, t) at position $(di + j)b + t$; the required output puts it at $(dj + i)b + t$. We use base-two Shannon entropy. For a random input X, its deficit is $D_X = P - H(X)$. If X is uniform on a nonempty set F, this is $D = P - \log_2 |F|$. A helper Z is a deterministic function of the whole input, supplied to the participants specified in a particular model. If X is supported on F, the bound $|Z(F)| \le 2^g$ implies $H(Z) \le g$. Each theorem specifies access to the helper; it does not supply a tape implementation of that access.

The finite coding results allow arbitrary b unless stated otherwise. For arbitrary input laws they use D_X; the cardinality formula for D requires the uniform law on F. All uses of $o(P)$ are along the indicated size sequence.

Native machines. We use the unchanged model of [9]. A machine has $q + 1$ states, k two-way infinite work tapes over $g + 1$ symbols (one blank), a read-only two-way input tape, and a one-way write-only output tape. In one transition each input or work head moves by -1, 0 or 1; the machine may append at most one output bit. The input lies in cells $0, \dots, P - 1$, with an end marker elsewhere. All heads start at zero, work tapes are blank, and the output is empty. The transition table, state set, alphabet and tape count are finite and independent of the input for a uniform machine. Write $G = \max(1, \lceil \log_2(g + 1) \rceil)$ for the charge in bits per work cell. "Computes within t" means that an actual halting configuration with exactly the specified output is reached after at most t transitions.

Imported Lean: `Transpose.Statement`
`Transpose.TM`
`Transpose.ComputesWithin`

Imported Lean: `Transpose.Tape.DefsM`
`Transpose.Tape.cellBits`

Definition 2.1 (The unchanged decision target)**.** Let

$$b(d) = \max(1, \lceil \log_2 d \rceil), \qquad P(d) = d^2 b(d), \qquad d \ge 1.$$

Statement (T) asserts that there exist one native machine M and one constant $c \in \mathbb{N}$ such that, for every $d \geq 1$ and every input matrix, M halts with its raw transpose within $c(P(d) + 1)$ steps.

Imported Lean: `Transpose.Statement`
`Transpose.entryBits`
`Transpose.inputLength`
`Transpose.encode`
`Transpose.transpose`
`Transpose.TransposesWithin`
`Transpose.LinearTimeTransposable`

The Lean files use n for the dimension in many declarations; the manuscript consistently writes d. Formally, the width is $\mathrm{Nat.log2}(d-1)+1$, including the convention at $d = 1$. The original target and checker are reused without alteration; (T) remains open.

The multiplication connection (cited background). Write $\lg x = \max(1, \lceil \log_2 x \rceil)$ and let $\lg^* x$ be the number of iterations of this rounded logarithm needed to reach 1. Harvey and van der Hoeven [17, Corollary 6.9] reduce transposition with entry width b to multiplication with an iterated-logarithm factor $\ell_0 = \max(1, \lg^* d - \lg^* b)$. At $b = \lg d = b(d)$ this factor is one. The encoding conventions are reconciled in the earlier paper's discussion of its Proposition on (T) and 1-logarithmic transposition. Consequently a negative resolution of (T) would exclude linear-time multiplication in the multitape model. A positive resolution of (T) would not by this implication produce a multiplier. For binary entries $b = 1$, the cited reduction from a linear multiplier instead gives an $O(P \log^* P)$ transposer.

Scope of the sequence statements. Native exclusion theorems below can allow a machine chosen separately for each size, while fixing its number of work tapes and cell-bit charge and bounding the logarithm of its state count by $o(P)$. Such statements are stronger than their fixed-machine special cases, but do not allow input-dependent advice without charging it. The correctness set may have deficit $o(P)$; geometric hypotheses refer to actual runs on that set. The required universal and subsequential quantifiers are given explicitly in the motion section.

3 Related work and attribution

3.1 The shared tape model and the scope of the question

This paper continues Brende and Claude Opus 5.5's *Matrix Transposition and the Tape Coding Conjecture* [9]. We use its native machine model, canonical entry width, cut inequality, and limiting path condition. These are imported directly from the original source package, together with its locality, counting and pass bounds and its reduction to causal coding on tape networks.

Harvey and van der Hoeven's reduction relates multiplication to transposition in the multitape model [17]. Its coefficient-width qualification matters here: their binary-matrix consequence of linear-time multiplication has an iterated-logarithm loss, whereas their larger-coefficient results yield the linear-time implication used for the canonical logarithmic entry width. We do not replace the binary conclusion by a linear bound. Classical transposition lower bounds for one-work-tape machines, including the version with a separate output tape, concern restricted models [14, 13]; they do not establish the unrestricted fixed-multitape lower bound sought here.

Network coding has also been used to derive conditional multiplication lower bounds [1]. The general assertion that coding never improves undirected multiple-unicast throughput has since been

refuted [8, 32]. Applying such a code to transposition would still require a tape implementation with all preparation and access costs charged.

3.2 Local coding, nonlinear recoding, and helper information

Index coding provides the natural comparison for receivers that combine a public message with their original rows. Bar-Yossef, Birk, Jayram, and Kol relate index-code length to minrank and acyclic subgraphs [3]; Blasiak, Kleinberg, and Lubetzky develop entropy-based bounds for broadcast rate [4]. We also use established entropy methods, including Shearer's covering inequality [11], and bounds obtained by counting code fibers.

Distributed and multi-sender index coding are closer comparisons because sender knowledge constrains the permitted encoders. Sadeghi, Arbabjolfaei, and Kim study distributed servers holding prescribed message subsets [27]. Li, Ong, and Johnson develop cooperative composite coding and a sender-constrained rank-minimization formulation [19, 20]. Our fixed original-row packet models can be interpreted in this setting after specifying the entry-level messages and column demands. Adaptive queries additionally choose their access supports from the public history; a fixed sender allocation does not by itself express this interface.

The adaptive separator theorem addresses the following interface: each query has specified access to *original* rows, its support and subsequent queries may depend on the public history, and the correct-input law can be correlated and nonuniform. Its graph records which rows a query accesses jointly; it is not the receiver side-information graph of standard index coding. The theorem charges both the helper and source entropy deficit and allows a separator chosen from the completed public history. The original-row access and public-history hypotheses distinguish this statement from the cited index-coding bounds.

Even multi-sender results require care. In the single-uniprior model of Ong, Ho, and Lim [25], each receiver initially knows one message, known to no other receiver, and its demands consist of whole messages. Bundling each row into a message changes our column demands into partial-message demands; splitting the rows into entries changes the single-uniprior hypothesis. Thus that characterization cannot simply be substituted for the original-row locality argument.

Our second family permits arbitrary bijective nonlinear recoding within rows but restricts the subsequent packets to fixed single-coordinate or pairwise XORs. This restriction is substantial: nonlinear index codes can outperform linear ones [21]. The component-flip and graph-distance arguments apply to this fixed-packet architecture. Similarly, the two-stage helper theorem counts fibers of a specific triangular–causal composition. Its helper selects the whole composition; no additional uncharged global operation is allowed between the stages.

3.3 Affine geometry, expanders, and paid computation

There is a longstanding distinction between simple one-dimensional affine constructions and expansion. Klawe ruled out several proposed expander constructions, including one-dimensional analogues of the Gabber–Galil construction [18]. Our affine-motion theorem has a different statement: it concerns countably many local affine branches of normalized head paths, measures exceptional time, and applies the original limiting cut condition. The approximation version retains the stated subsequence and bounded-cover hypotheses. We do not identify these conclusions with Klawe's finite-graph theorem.

Monotone expanders and their connections to page and pushdown graphs were studied by Dvir and Wigderson [15]. Bourgain and Yehudayoff constructed a continuous monotone expander using Möbius transformations and proved the associated growth and restricted spectral-gap results [7]. Our

analytic development follows this established work, proving the rational-family, growth, flattening and spectral statements needed for the native implementation.

The scalar sum-product and projection ingredients belong to Bourgain's discretized ring and projection theory [5, 6]. Their mathematical origin is distinct from the formal source we reuse. The pinned OpenAI library [26] supplies more than the endpoint `scalar_weak_radius_projection`: we directly import `ScalarDilationBound`, `ScalarPluennecke`, `ScalarBallCandidates`, `ScalarCoverCount`, and `ScalarWeakRadiusProjection`. The bridge also uses its scalar-cell, ball-mass, zoom, and ratio-family definitions and estimates, including `scalar_zoom_cells`, `scalar_zoom_ball_bound`, `scalar_refined_ball_bound`, `scalar_ratio_family_ball`, and `scalar_ratio_family_card`. The local arguments verify the hypotheses and derive the expansion inputs. The bibliography pins the formal source revision, whose imported proofs are checked in the same dependency chain.

The padding theorem adds a computational conclusion: the walk is generated, executed, cleaned up, and followed by the original computation at additive linear cost, with the stated fixed-machine and resource quantifiers. These costs do not follow merely from a graph's expansion or a formula for its maps. The scan-palette and recursion exclusions also account for actual execution, under the stated affine regularity and exceptional-time hypotheses.

3.4 The transpose prediction conjecture is already false

Conjecture 36 of Mittal and Raz [24] proposed an exponentially small success bound for a matrix-transpose prediction game. Its failure is explicitly recorded in Mittal's dissertation [23, §10.8.2, Proposition 10.8.3 and Lemma 10.8.5], which credits a proof of Braverman and Forbes from their 2022 personal communication. We cite the dissertation as the accessible proof and claim no new refutation.

Related counterexamples also follow from earlier coded-caching constructions: placement delivery arrays of Yan et al. [31], induced-matching schemes of Shanmugam, Tulino, and Dimakis [28], and the projective-geometric schemes of Chittoor et al. [10]. These papers do not themselves state a disproof of Conjecture 36. Their induced-matching classes give XOR prediction strategies with success probability 2^{-r}, where r is the number of nonempty classes. Suitable square conversions give $o(d)$ observed rows per player and $r = o(d^2)$. For the dense-graph route, the construction of Alon, Moitra, and Sudakov [2] is combined with minimum-degree pruning. This avoids a fixed-vertex degree estimate in the supplied v1 caching preprint. The conversions and version-specific caveat are detailed in `paper/prior-art.txt`; they are background arguments, not new Lean results claimed here.

The game does not charge for acquiring or combining the observed rows. Its counterexamples close that prediction-game route to a lower bound but do not supply a causal tape implementation.

3.5 Formal dependencies and novelty boundaries

The formal proofs use Lean 4 [12] and the pinned Mathlib4 library [29, 30], in addition to the two explicit source dependencies above. Standard finite-set, entropy, measure, topology, and arithmetic results are reused. We distinguish proving the present endpoints from claiming new underlying analytic or information-theoretic principles.

The literature comparison covers the cited primary sources and targeted searches for local index coding, two-stage causal maps, affine motion and recursive tape schedules. It does not establish exhaustive priority. We claim the stated local-coding bounds, motion consequences and native implementations, while crediting their analytic and information-theoretic ingredients. Supporting lemmas are included for their role in these arguments, without a blanket novelty claim.

4 Local coding obstructions

The construction programme of [9] needs coding stages that transmit fewer bits than the uncoded data. To test a proposed stage, one must specify what each operation reads, what each receiver knows, and what input-dependent information selects the operations. This section gives three obstructions: a separator bound for adaptive queries of original rows; a distance bound for fixed coordinate-XOR broadcasts after bijective row recoding; and a helper-or-deficit bound for two ordered nonlinear stages.

Each result concerns its stated finite coding model. None asserts that every fast tape transposer has that form. In particular, the first model reads original rows, not stored rows of coded workspace.

Throughout this section, $d, b \in \mathbb{N}$, $[d] = \{0, \dots, d-1\}$,

$$\mathcal{X} = \{0,1\}^{[d]\times[d]\times[b]}, \qquad P = d^2 b.$$

A row belongs to $Q = \{0,1\}^{[d]\times[b]}$, and $(x^{\mathsf{T}})_{ijt} = x_{jit}$. The finite-coordinate adapter in `MatrixView` uses the transpose and encoding of the first paper; it does not replace their specification. Except where positivity is stated, zero dimensions and zero entry widths are permitted.

For a probability law w on a finite correct-input family $F \subseteq \mathcal{X}$, write $X \sim w$ and

$$H(X) = -\sum_{x\in F} w(x)\log_2 w(x), \qquad D_w = P - H(X),$$

with the usual zero-mass convention. The weights may vanish and need not be independent across rows, entries, or bits. Under the uniform law on nonempty F, write $D = P - \log_2 |F|$. All entropies in this section are in bits; the Lean entropy functional uses natural logarithms and is divided by $\log 2$ at the endpoints.

4.1 Adaptive queries of original rows

Definition 4.1 (Public original-row strategy)**.** A local query consists of a support $A \subseteq [d]$ and a function $q : \mathcal{X} \to \{0,1\}$ such that

$$x|_A = y|_A \quad \Longrightarrow \quad q(x) = q(y).$$

The support may have any finite size. A public strategy assigns, to every finite binary history h, either a local query or a halt instruction. When it selects q, the next transmitted bit is $q(x)$ and the history becomes $h\,q(x)$. An execution is complete only if the strategy then halts; the halt instruction adds no bit.

A helper $z : \mathcal{X} \to \mathcal{Z}$, with $\mathcal{Z}$ finite, selects a strategy $s_{z(x)}$. Receiver j has a decoding function of precisely $z(x)$, its own original row x_j, and the completed transcript $M(x)$. For a declared family F, the correctness requirement is that, for every $x \in F$, the selected strategy terminates with transcript $M(x)$ and every receiver returns $(x^{\mathsf{T}})_j$. There is no termination or correctness requirement outside F.

Lean: `TransposeFollowup.ProtocolTruncation`
`TransposeFollowup.PublicStrategy`
`TransposeFollowup.PublicStrategy.Executes`

Lean: `TransposeFollowup.LocalProtocol`
`TransposeFollowup.LocalQuery`

Thus the helper and public history determine the next support and the stopping decision. Any further input-dependent control must be conveyed by query answers. Query evaluation and decoding are unrestricted; the cost is the number of transmitted bits, including those used for control.

For a completed transcript m under helper value z, let $G(z,m)$ be the simple graph on $[d]$ obtained by completing each queried support to a clique. Only the queries actually encountered along m contribute edges. In Lean, this graph is obtained from the strategy truncated at $|m|$; the execution hypothesis ensures that no query of the completed run is lost. A separator selection is any function $S(z,m) \subseteq [d]$. No balance condition is assumed. If $\mathcal{C}(G-S)$ denotes the components of the induced graph outside S, define

$$\mathcal{L}(G,S) = db|S| + b \sum_{C \in \mathcal{C}(G-S)} |C|^2. \tag{1}$$

For random input X, abbreviate $Z = z(X)$, $G_X = G(Z, M(X))$, and $S_X = S(Z, M(X))$.

Theorem 4.2 (Adaptive separator bound)**.** *For the strategy of Definition 4.1, correct on F, and every probability law w on F,*

$$\mathbb{E}_w|M(X)| \;\geq\; P - \mathbb{E}_w\mathcal{L}(G_X, S_X) - 2D_w - 2H(Z). \tag{2}$$

The separator may depend arbitrarily on the helper and completed transcript. Correctness is required on all of the declared family F, including its zero-probability inputs. For uniform X on nonempty F, the same statement holds with $D_w = D$.

Lean: `TransposeFollowup.GraphSeparatorBound`
`TransposeFollowup.graph_expected_separator_bound_of_distribution`
`TransposeFollowup.graph_expected_separator_bound`

The bound excludes a fixed fractional saving whenever expected separator loss, input deficit, and helper entropy are all $o(P)$. It therefore tests a proposed coding stage without imposing linearity on its queries or decoders.

Outline of the formal proof. First replace the possibly infinite strategies by finite protocols, truncating all of them at the maximum transcript length attained on F. Finiteness of F makes this maximum finite, and the truncation preserves every specified execution, its cost, and its queried supports. This step imposes no assumption on the original strategies outside F.

Fix a helper value z, transcript m, and separator assignment u attained on F. Let R be the *virtual full fiber*: all inputs producing m in the fixed finite protocol for z and having separator assignment u. Membership in R does not require the input's actual helper to be z, nor does it require correctness. Every query along m meets at most one component of $G(z,m) - S(z,m)$ outside the separator. Consequently, component assignments taken from different elements of R can be combined freely while keeping u fixed. The locality equation for each query shows, one step at a time, that the combined input follows the same transcript. Thus R is exactly the product of its component projections, with the separator coordinates fixed.

Let A be a nonempty correct subfamily with this helper, transcript, and separator assignment. No product structure is asserted for A. On A, each component's original rows determine the columns requested by its receivers. Fixed separator rows likewise determine the separator receivers' columns, so those rows and columns have zero entropy.

For the remaining coordinates, each component's owned-and-demanded set covers an intercomponent bit twice and an internal bit once. Adding one copy of each internal bit gives a twofold entropy cover. Decoding reduces each owned-and-demanded entropy to the owned-row entropy. The product structure of R bounds the sum of these entropies by $\log_2 |R|$; the internal bits contribute at most $b\sum_C |C|^2$. Under the uniform law on A, this gives

$$2\log_2 |A| \leq \log_2 |R| + b \sum_{C \in \mathcal{C}(G-S)} |C|^2. \tag{3}$$

The product property is used only for R, not for the correlated family A.

For the aggregation, append the separator rows, in a fixed order, to each transcript. A leaf label a consists of the helper and this augmented transcript. Write f_a for its correct-fiber size, V_a for its virtual full-fiber size, $q_a = |m| + db|S|$ for the augmented length, and $r_a = b\sum_C |C|^2$ for the internal loss. For each fixed helper z, the virtual fibers are disjoint, and the augmented transcripts form a prefix-free code. The latter follows because completed public transcripts are prefix-free and, once m is known, the appended row-string length is fixed. Hence

$$\sum_{a:\, z(a)=z} V_a \le 2^P, \qquad \sum_{a:\, z(a)=z} 2^{-q_a} \le 1.$$

Let A_* be the random leaf label. Conditional Gibbs inequalities for these two budgets and the finite-fiber entropy inequality give

$$\begin{aligned} H(X) &\le H(A_*) + \mathbb{E}\log_2 f_{A_*}, \\ H(A_*) &\le H(Z) + P - \mathbb{E}\log_2 V_{A_*}, \\ H(A_*) &\le H(Z) + \mathbb{E}q_{A_*}. \end{aligned}$$

Adding the last two inequalities and using twice the first together with (3) yields $2H(X) \le P + 2H(Z) + \mathbb{E}(q_{A_*} + r_{A_*})$. Since $q_{A_*} + r_{A_*} = |M(X)| + \mathcal{L}(G_X, S_X)$, rearrangement proves (2). All averaging uses the supplied law; zero-mass fibers contribute zero.

Corollary 4.3 (Worst-case cost and attained helper range)**.** *Under the same strategy assumptions, let $F \neq \varnothing$ and let $B, L, g \in \mathbb{N}$. If every $x \in F$ satisfies $|M(x)| \le B$ and $\mathcal{L}(G_x, S_x) \le L$, and $|z(F)| \le 2^g$, then*

$$B \ge P - L - 2(D + g).$$

Only helper values attained on F are charged; the ambient helper alphabet is still required to be finite.

Lean: `TransposeFollowup.GraphSeparatorBound`
`TransposeFollowup.graph_worst_case_separator_bound`

This form gives a direct rejection test from worst-case bounds on a candidate's broadcast, separator loss, and attained helper range.

Outline of the formal proof. Use the uniform law in Theorem 4.2, bound both expectations by their pointwise bounds, and use $H(Z) \le \log_2 |z(F)| \le g$.

Theorem 4.4 (Separate sharpness of the two information charges)**.** *In the stated original-row model, the coefficient of D and the coefficient of $H(Z)$ in (2) cannot separately be replaced by a constant smaller than 2, with the other terms unchanged. Each assertion has a zero-transcript witness with $d = 2$, empty separator, and every $b > 0$.*

Lean: `TransposeFollowup.SeparatorSharpness`
`TransposeFollowup.SeparatorSharpness.deficit_coefficient_at_least_two`
`TransposeFollowup.SeparatorSharpness.helper_coefficient_at_least_two`

These witnesses establish optimal universal coefficients, not sharpness when $d \to \infty$ and $D, H(Z) = o(P)$: each has $d = 2$ and its nonzero information charge equals $P/4$. Improvements restricted to the small-deficit, small-helper regime remain possible.

Outline of the formal proof. The empty interaction graph has two singleton components, so its loss is $2b$, while $P = 4b$. For the deficit coefficient, take the family $x_{01} = x_{10}$, of cardinality 2^{3b}, and a constant helper. Each receiver already has its column in its own row. The transcript has

length zero, $D = b$, and a coefficient c would require $0 \geq 2b - cb$. For the helper coefficient, take the full input cube and the helper $Z = x_{01} \oplus x_{10}$. Each receiver recovers the other off-diagonal entry from its own row and Z, again with no transcript. Here $D = 0$ and, under the uniform input law, $H(Z) = b$. The same inequality forces $c \geq 2$.

4.1.1 A treewidth consequence

Use ordinary finite tree decompositions, with the natural-valued convention $\mathrm{tw}(\varnothing) = 0$: $\mathrm{tw}(G)$ is the least $t \in \mathbb{N}$ for which a tree decomposition has bags of size at most $t + 1$. The decomposition tree has arbitrary finite shape.

Lemma 4.5 (Integral fragmentation)**.** *For every finite simple graph G on d vertices and every $L \in \mathbb{N}$, there is $S \subseteq V(G)$ such that*

$$(L + 1)|S| \leq (\mathrm{tw}(G) + 1)d, \qquad |C| \leq L \quad (C \in \mathcal{C}(G - S)).$$

For a graph on $[d]$, a separator can consequently be chosen so that

$$(L + 1)\mathcal{L}(G, S) \leq bd\big((\mathrm{tw}(G) + 1)d + L(L + 1)\big).$$

Lean: `TransposeFollowup.Treewidth`
`TransposeFollowup.treewidth_fragmentation`
`TransposeFollowup.treewidth_separator_loss`

Outline of the formal proof. Assign each graph vertex to one bag that contains it, giving integer weights of total d to the decomposition tree. The weighted-tree fragmentation lemma removes a set of tree vertices whose number, multiplied by $L + 1$, is at most d, leaving weight at most L in each remaining tree component. Remove their bags from G. The running-intersection and edge-cover properties transfer the component bound to G, while each removed bag costs at most $\mathrm{tw}(G) + 1$ vertices. Finally, $\sum_C |C|^2 \leq L \sum_C |C| \leq Ld$ gives the loss estimate. This is a standard tree-decomposition argument; its role here is to provide the exact integral budget used in the next optimization.

Corollary 4.6 (Savings force large average treewidth)**.** *Assume $d, b > 0$ and the strategy hypotheses of Theorem 4.2. For any probability law w on F, suppose $\mathbb{E}_w|M(X)| \leq (1 - \varepsilon)P$, where $\varepsilon \in \mathbb{R}$. Put*

$$\eta = \frac{D_w + H(Z)}{P}, \qquad a = \max\{\varepsilon - 2\eta, 0\}.$$

Then

$$\mathbb{E}_w \,\mathrm{tw}(G_X) \geq \frac{d}{4}a^2 + \frac{a}{2} - 1. \tag{4}$$

Lean: `TransposeFollowup.AdaptiveTreewidth`
`TransposeFollowup.average_treewidth_lower_bound_of_distribution`
`TransposeFollowup.average_treewidth_lower_bound`

Outline of the formal proof. For each integer $L \geq 0$, choose the separator supplied by Lemma 4.5. It is a function of the public leaf graph, so it is an allowed separator. Average its loss bound and apply the separator theorem. With $E = \mathbb{E}\,\mathrm{tw}(G_X)$ and $a_0 = \varepsilon - 2\eta$, the resulting family of tests is

$$d(L + 1)a_0 \leq d(E + 1) + L(L + 1) \qquad (L \in \mathbb{N}).$$

If $a_0 \le 0$, nonnegativity of treewidth suffices. Otherwise set $r = da_0/2$ and $L = \lfloor r \rfloor$. The elementary identity

$$(L+1)(2r-L) - (r^2+r) = (r-L)(L+1-r) \ge 0$$

gives $d(E+1) \ge r^2 + r$ from the test at this L, which is (4).

As $d \to \infty$, with $b > 0$ and $D_w + H(Z) = o(P)$, a fixed fractional saving therefore requires average treewidth $\Omega(d)$. Alternatively, separators of size $o(d)$ whose remaining components all have size $o(d)$ have loss $o(P)$, provided these estimates hold uniformly over the completed runs. Such original-row strategies cannot yield a fixed fractional saving. Graphs without such separators remain candidates for this model.

The first paper's construction discussion asks for locally encoded index coding [9, section "The construction side"]. The separator bound gives an explicit obstruction for hyperfinite interaction graphs. If, for each $\varepsilon > 0$, every completed-run graph has a separator of at most εd vertices leaving components of size at most $K(\varepsilon)$, uniformly in d and the run, then

$$\mathcal{L}(G, S) \le \varepsilon P + bK(\varepsilon)d.$$

Together with $D_w + H(Z) = o(P)$, this excludes a fixed fractional saving. The treewidth consequence also excludes such savings whenever $\mathbb{E}_w \operatorname{tw}(G_X) = o(d)$, without the uniform component-size bound required by hyperfiniteness. For a fixed interaction graph Γ_d, it suffices that $\operatorname{tw}(\Gamma_d) = o(d)$.

These statements allow nonlinear queries, public adaptivity, helpers, and correlated correct sets. They leave the coding problem on bounded-degree expanders open and do not strengthen Theorem P or prove the Tape Coding Conjecture. They also apply to finite-field data after an explicit binary encoding: query answers and costs must be counted in bits, and any deficit from restricting to valid encodings must be retained.

4.2 Bijective row recoding and fixed XOR packets

The preceding separator argument leaves highly connected row graphs open. For a more restrictive broadcast format, an independent obstruction applies even when the row graph has bounded degree and large treewidth.

Definition 4.7 (Recoded fixed-XOR code)**.** For each $i \in [d]$, choose an arbitrary bijection $A_i : Q \to Q$ and write $y_i = A_i(x_i)$. Fix $B \in \mathbb{N}$ packets, each either one coordinate y_{ijt} or the XOR of two coordinates of y. Packet choices are fixed before the input is supplied. For a simple graph G on $[d]$, locality means that the row indices of a pair packet are equal or adjacent in G; singleton packets impose no additional edge condition. Receiver j receives the complete B-bit broadcast and its own *original* row x_j, and must recover $(x^{\mathsf{T}})_j$ for every $x \in \mathcal{X}$.

Lean: `TransposeFollowup.RecodedXor`
`TransposeFollowup.RecodedXorCode`
`TransposeFollowup.RecodedXorCode.Correct`
`TransposeFollowup.RecodedXorCode.LocalTo`

Lean: `TransposeFollowup.XorPackets`
`TransposeFollowup.XorPacket`
`TransposeFollowup.XorSystem`

The bijections and the decoders need not be linear. Repeated packets, same-row pairs, and self-XORs are allowed. A self-XOR is a permitted constant zero packet. There is no helper in this model, and correctness is on the entire input cube, not merely a high-entropy subfamily.

For $\ell \in \mathbb{N}$, call (i, j) *ℓ-near* if there exists a walk in G from i to j of length strictly below ℓ, and let

$$Q_\ell = b\,\big|\{(i,j) \in [d]^2 : (i,j) \text{ is } \ell\text{-near}\}\big|.$$

This walk-based definition excludes disconnected pairs; it does not assign them distance zero. In particular $Q_0 = 0$.

Theorem 4.8 (Distance bound after arbitrary row recoding)**.** *Every correct code of Definition 4.7 that is local to G satisfies, for all $\ell \in \mathbb{N}$,*

$$(\ell+1)P \le (\ell+1)B + P + \ell Q_\ell, \qquad B \ge P - \frac{P + \ell Q_\ell}{\ell+1}. \tag{5}$$

No positivity or connectedness hypothesis is needed for the integral form.

Lean: `TransposeFollowup.XorComponentCount`
`TransposeFollowup.RecodedXorCode.broadcast_lower_bound_scaled`
`TransposeFollowup.RecodedXorCode.broadcast_lower_bound`

Small graph neighborhoods limit the savings even after nonlinear row recoding. The bounded-degree corollary below adds an obstruction on expanders. The path/cycle corollary illustrates the distance argument, but the earlier separator bound gives a better constant on those graphs.

Outline of the formal proof. Form the variable graph on the P transformed bit coordinates: distinct endpoints of a pair packet are adjacent. A coordinate appearing as a singleton packet is an anchor. Let u be the number of components containing no anchor. The broadcast together with one transformed bit from each unanchored component determines every transformed bit: equal broadcasts make the difference of two assignments constant on each component, and anchors or chosen representative bits determine that constant. Injective finite counting therefore gives $P \le B + u$.

An unanchored component can be flipped simultaneously without changing the broadcast. Pull this flip back through the row bijections. If its represented row set is R, original rows outside R do not change. Their receivers have the same side information and broadcast before and after the flip, so all columns indexed outside R also stay fixed. The raw change is therefore confined to $R \times R \times [b]$. This holds at every basepoint, despite nonlinearity of the row bijections.

If the variable component contains at most ℓ coordinates, any two of its coordinates are joined by a simple path of length below ℓ. Projection to row indices collapses same-row edges and maps other edges to edges of G. Thus its raw changes are confined to the Q_ℓ near bits. Let u_{small} count such unanchored components. Their disjoint nonempty flips generate $2^{u_{\text{small}}}$ distinct transformed assignments and hence the same number of raw assignments. Because each flip fixes every far bit at every basepoint, their compositions do so as well. The orbit fits into Q_ℓ binary coordinates, giving $u_{\text{small}} \le Q_\ell$.

Disjointness and nonemptiness of all unanchored components imply

$$(\ell+1)u \le P + \ell u_{\text{small}}.$$

Indeed, a large component pays at least $\ell+1$ coordinates, while a small one pays at least one and receives the additional allowance ℓ. Combining these inequalities proves (5).

Corollary 4.9 (Bounded degree)**.** *For a correct code local to G, if $d \ge 2$ and G has maximum degree at most $\Delta \in \mathbb{N}$, then*

$$B \ge P - \frac{6\log(\max\{2,\Delta\})}{\log d}\,P.$$

The logarithm base is immaterial to this ratio. For fixed Δ and $b > 0$, no code in this model achieves a fixed positive fractional saving as d grows.

Lean: `TransposeFollowup.XorDegreeBound`
`TransposeFollowup.RecodedXorCode.fixed_degree_broadcast_lower_bound`

Outline of the formal proof. Put $\delta = \max\{2, \Delta\}$ and $k = \lfloor \lfloor \log_\delta d \rfloor / 2 \rfloor$. Counting walks bounds each radius-k ball by $2\delta^k$, so $Q_{k+1} \le 2bd\delta^k$. The integer estimate $(k+1)\delta^k \le d$ yields $(k+2)P \le (k+2)B + 3P$ from the distance theorem. Finally $\log d \le 2(k+2)\log\delta$ converts this to the displayed bound. The argument is uniform in b and does not assume regularity or connectedness of G.

Corollary 4.10 (Paths, cycles, and their embedded subgraphs)**.** *For a correct code local to G, assume $d > 0$. If there is an injective graph homomorphism from G into a cycle on $m \ge 1$ vertices, then*

$$B \ge P - bd\left(\sqrt{8(d+3)} - 4\right). \tag{6}$$

In particular this holds for paths, cycles, and their subgraphs, with arbitrary vertex labels. The same bound holds whenever every ℓ-near ball has at most $2\ell - 1$ vertices for every integer $\ell \ge 1$.

Lean: `TransposeFollowup.XorPathCycleBound`
`TransposeFollowup.RecodedXorCode.linear_ball_broadcast_lower_bound`
`TransposeFollowup.RecodedXorCode.cycle_embedding_broadcast_lower_bound`
`TransposeFollowup.RecodedXorCode.subcycle_broadcast_lower_bound`
`TransposeFollowup.RecodedXorCode.cycle_broadcast_lower_bound`
`TransposeFollowup.RecodedXorCode.path_broadcast_lower_bound`

For these graphs, the saving is at most $O(P/\sqrt{d})$, improving the $O(P/\log d)$ error supplied by the bounded-degree estimate.

Outline of the formal proof. An injective graph homomorphism sends a near ball into the corresponding cycle ball. Walking fewer than ℓ steps in a cycle gives at most $2\ell - 1$ possible integer offsets; collisions can only reduce this number. Hence $Q_k \le bd(2k-1)$ for every $k \ge 1$. The distance theorem gives

$$P - B \le bd\,\frac{d + k(2k-1)}{k+1}.$$

Choose $k = \lfloor \sqrt{\lfloor (d+3)/2 \rfloor} \rfloor$, put $j = k+1$, and let $t = \sqrt{(d+3)/2}$, where the quotient defining t is real. Then $k \le t < j$, $j \ge 2$, and

$$\frac{d + k(2k-1)}{k+1} = 4t - 5 + \frac{2(j-t)^2}{j} \le 4t - 4 = \sqrt{8(d+3)} - 4.$$

The estimate includes $d = 1$.

Every recoded fixed-XOR code also gives an original-row strategy: transmit its fixed packets in order, with a constant helper and correctness on the full cube. Its interaction graph is a subgraph of G, so the separator bound gives $B \ge P - \mathcal{L}(G, S)$ for every S. On a path or cycle, about $\sqrt{d}$ evenly spaced separator vertices leave components of size about $\sqrt{d}$, giving loss $(2 + o(1))bd^{3/2}$ instead of the leading $2\sqrt{2}\,bd^{3/2}$ in Corollary 4.10. For example, on the 100-cycle, ten separator vertices leave ten components of size nine, giving loss $1810b$, whereas (6) gives about $2471b$. The additional reach of the distance method is therefore its bounded-degree conclusion on graphs, including expanders, for which small separators are unavailable.

The bounded-degree conclusion applies even to expanders, but only for the specified packet format. It does not answer the first paper's question about *arbitrary* linear functionals supported on two rows: one such functional may involve many coordinates, and a single common row bijection need not convert a whole family of them into pairwise coordinate XORs. One sufficient condition is that, for each row, the distinct nonzero row-parts of all binary-linear packets are linearly independent.

Extend those forms to a basis of the row's dual space and use that basis as its linear recoding; every nonzero packet then becomes a singleton or a two-coordinate XOR. Zero packets can be omitted. This condition is not necessary for the full model, which also permits same-row pairs: the packets u_1, u_2, and $u_1 \oplus u_2$ already have the required format although their row-parts are dependent. Linearity over GF(2), or over an extension field, does not by itself imply membership; the bit-expanded packets must admit the specified common recoding and packet format. The first paper's XOR-frame model concerns changes of address frame in a gadget, a different restriction. The bounded-degree corollary rules out a fixed fractional saving through independent bijective row preprocessing followed by a fixed coordinate-XOR broadcast local to a bounded-degree graph.

4.3 Two nonlinear ordered stages and a global helper

A different construction strategy lets later stages act on coded rows. The following model allows two coupled stages on one bank of d rows. Its obstruction is independent of runtime: all allowed maps are granted as primitives.

Definition 4.11 (Triangular and causal row maps)**.** For a finite row alphabet Q and a permutation σ of $[d]$, a triangular row map has the form

$$T(u)_i = \phi_{i,u_{<i}}(u_i), \qquad u_{<i} = (u_j)_{\sigma(j)<\sigma(i)},$$

where $\phi_{i,v} : Q \to Q$ is a bijection for every fixed earlier-row tuple v. Dependence on v is arbitrary. A causal row map in an order τ has the form

$$C(p)_j = \gamma_j\big((p_l)_{\tau(l)\le\tau(j)}\big),$$

with arbitrary functions γ_j; no injectivity is required. These formulas refer to the incoming row tuple of each stage.

Lean: `TransposeFollowup.TriangularRows`
`TransposeFollowup.TriangularRows`
`TransposeFollowup.TriangularRows.apply`
`TransposeFollowup.CausalRows`
`TransposeFollowup.CausalRows.apply`

Definition 4.12 (Two-stage branch and helper selection)**.** A two-stage branch consists of three row permutations $\pi_{\mathrm{in}}, \pi_{\mathrm{mid}}, \pi_{\mathrm{out}}$, arbitrary local maps $a_i, c_i : Q \to Q$, a triangular map T, local bijections $\lambda_i : Q \to Q$, and a causal map C. It acts by

$$u_i = a_i(x_{\pi_{\mathrm{in}}(i)}), \qquad v_i = T(u)_i, \qquad w_i = \lambda_i(v_i),$$
$$p_j = w_{\pi_{\mathrm{mid}}(j)}, \qquad q_j = C(p)_j, \qquad \Phi(x)_i = c_i(q_{\pi_{\mathrm{out}}(i)}).$$

A helper-selected architecture is a function $z : \mathcal{X} \to \mathcal{Z}$ and a branch Φ_z for each $z \in \mathcal{Z}$. Here $\mathcal{Z}$ may be infinite. For $Q = \{0,1\}^{[d]\times[b]}$, correctness on F means $\Phi_{z(x)}(x) = x^{\mathsf{T}}$ for every $x \in F$.

Lean: `TransposeFollowup.TwoStageRows`
`TransposeFollowup.TwoStageRows`
`TransposeFollowup.TwoStageRows.apply`

Lean: `TransposeFollowup.TwoStageHelper`
`TransposeFollowup.TwoStageHelper`
`TransposeFollowup.TwoStageHelper.CorrectOn`

All maps and both stage orders may depend on the single helper value. Once that value is fixed, they are fixed functions of the displayed inputs. The first stage is required to be bijective in its own row argument, and the middle local maps are bijections. The input maps, output maps, and second stage may discard information. An extra bank, a global operation between the stages, or another input-dependent controller is not part of this definition.

Lemma 4.13 (Block injectivity)**.** *A triangular map T is block-injective: for every $I \subseteq [d]$,*

$$u|_{I^c} = u'|_{I^c}, \qquad T(u)|_I = T(u')|_I \quad \Longrightarrow \quad u = u'.$$

It is a bijection on the finite row space. Postcomposition by rowwise bijections preserves block injectivity, and the inverse of a block-injective bijection is block-injective. In particular, the inverse of $W(u)_i = \lambda_i(T(u)_i)$ has this property.

Lean: `TransposeFollowup.TriangularRows`
`TransposeFollowup.TriangularRows.block_injective`
`TransposeFollowup.TriangularRows.equiv`
`TransposeFollowup.blockInjective_rowwise`
`TransposeFollowup.blockInjective_inverse`

Lean: `TransposeFollowup.TwoStageRows`
`TransposeFollowup.TwoStageRows.inverse_block_injective`

This supplies the recovery property used in the next counting lemma directly from the allowed row maps, without imposing it as an extra hypothesis.

Outline of the formal proof. Induct on the triangular order. Outside I the two incoming rows already agree. Inside I, earlier rows agree by induction, so the same bijection $\phi_{i,u_{<i}}$ is applied to both own-row inputs; equality of the outputs forces equality of those inputs. Taking $I = [d]$ gives global injectivity, and finiteness gives bijectivity. Rowwise postcomposition is cancelled one row at a time. For the inverse statement, apply block injectivity of the forward map with I^c in place of I.

Lemma 4.14 (A record for one branch)**.** *If one fixed two-stage branch is correct on an arbitrary family $F \subseteq \mathcal{X}$, then for every integer $0 \le r \le d$,*

$$|F| \le 2^{(d-r)db + r^2 b} = 2^{P - br(d-r)}. \tag{7}$$

The family may be empty or correlated.

Lean: `TransposeFollowup.TwoStageRows`
`TransposeFollowup.TwoStageRows.correct_family_card`

This bounds how many inputs one branch can handle; partitioning by the helper will then bound both its required range and its entropy.

Outline of the formal proof. Let τ be the second-stage order. Its last r incoming rows determine sets

$$I = \{i : \tau(\pi_{\mathrm{mid}}^{-1}(i)) \ge d - r\},$$
$$R = \pi_{\mathrm{in}}(I), \qquad J = \{i : \tau(\pi_{\mathrm{out}}(i)) \ge d - r\}.$$

All three sets have size r. The input rows u_I depend only on the original rows x_R. By causality, the final output rows outside J depend only on w_{I^c}.

Record w_{I^c} and the raw rectangle $x_{R \times J \times [b]}$. There are $2^{(d-r)db + r^2 b}$ possible records. To prove injectivity on F, suppose two correct inputs have the same record. Equality of w_{I^c} gives equality

of their final output rows outside J, hence equality of their raw columns outside J. The recorded rectangle supplies the remaining entries of the original rows in R, so those complete rows agree. Therefore u_I agrees. Block injectivity of W^{-1}, together with equality of w_{I^c}, now gives equality of all of w. The two complete outputs coincide; correctness and injectivity of transpose imply that the original inputs coincide.

The support sets come from the specified permutations and causal stage, as implemented in `FixedSupportSandwich`. The exponent identity uses $r \le d$.

Theorem 4.15 (Exact helper-or-deficit toll)**.** *For a helper-selected two-stage architecture correct on $F \subseteq \mathcal{X}$ and every integer $0 \le r \le d$,*

$$|F|\, 2^{br(d-r)} \le |z(F)|\, 2^{P}. \tag{8}$$

In particular,

$$|F|\, 2^{b\lfloor d^2/4 \rfloor} \le |z(F)|\, 2^{P}.$$

If $D \in \mathbb{R}$, $g \in \mathbb{N}$, $|F| \ge 2^{P-D}$, and $|z(F)| \le 2^{g}$, then

$$D + g \ge b\lfloor d^2/4 \rfloor. \tag{9}$$

The counting statements include $F = \varnothing$; the density premise in the last assertion implies $F \neq \varnothing$. Only the finite attained helper image is counted.

Lean: `TransposeFollowup.TwoStageHelper`
`TransposeFollowup.TwoStageHelper.correct_family_count_toll`
`TransposeFollowup.TwoStageHelper.quarter_count_toll`
`TransposeFollowup.TwoStageHelper.quarter_helper_bound_of_density`

A small helper range therefore forces a large correct-family deficit. For example, a branch architecture correct on the full cube needs at least $2^{b\lfloor d^2/4 \rfloor}$ attained helper values.

Outline of the formal proof. Partition F by its helper value. On each part the architecture is one fixed branch, so Lemma 4.14 bounds its cardinality by $2^{P-br(d-r)}$. Sum over the attained helper values and multiply by $2^{br(d-r)}$. Choosing $r = \lfloor d/2 \rfloor$ uses the integer identity $r(d-r) = \lfloor d^2/4 \rfloor$. The density conclusion follows by taking logarithms of positive quantities. No rounding of the real deficit D is needed, and no factorization of F is used.

Theorem 4.16 (Actual helper entropy under arbitrary input laws)**.** *Under the same architecture and correctness hypotheses, let X have any probability law on F and put $Z = z(X)$ and $D_w = P - H(X)$. Then, for every integer $0 \le r \le d$,*

$$D_w + H(Z) \ge br(d-r), \qquad D_w + H(Z) \ge b\lfloor d^2/4 \rfloor. \tag{10}$$

The ambient helper alphabet may be infinite. Its entropy is computed on the finite attained subtype. Uniform input gives the corresponding inequalities with $D_w = P - \log_2 |F|$.

Lean: `TransposeFollowup.TwoStageHelperEntropy`
`TransposeFollowup.TwoStageHelper.helper_entropy_deficit_bound_of_distribution`
`TransposeFollowup.TwoStageHelper.quarter_helper_entropy_deficit_bound_of_distribution`
`TransposeFollowup.TwoStageHelper.helper_entropy_deficit_bound`
`TransposeFollowup.TwoStageHelper.quarter_helper_entropy_deficit_bound`

Outline of the formal proof. Every fiber of the attained-helper map on F has size at most $C = 2^{P-br(d-r)}$ by the branch-count lemma. The shared finite-fiber entropy inequality gives

$H(X) \le H(Z) + \log_2 C$. In Lean this inequality follows from conditional Gibbs with comparison weight C^{-1} on each input of a helper fiber; each such fiber has comparison mass at most one. It is valid for arbitrary nonnegative weights, so specializing to a normalized law does not require positive mass at every input. Rearrangement gives the first inequality, and $r = \lfloor d/2 \rfloor$ gives the second.

For $d \to \infty$, $b > 0$, and $D_w = o(P)$, the entropy bound requires $H(Z) \ge (1/4 - o(1))P$. A construction in this model must therefore supply that much helper information even if its helper values are very unevenly distributed. The theorem does not cover more stages, extra banks, or an unrestricted causal network.

The first paper identifies a global oracle hint in the plain-versus-robust gap and argues heuristically that its proposed recursion needs savings at both encoder and decoder [9, sections "Plain versus robust" and "The construction side"]. The two-stage results quantify a helper-or-deficit toll for one explicit architecture; they neither remove the general hint nor strengthen the native pass lower bound. The three coding models in this section impose different restrictions. A construction escaping one obstruction must still be checked against the hypotheses of the others.

5 Affine motion and exceptional physical time

The motion obstruction concerns actual work-head trajectories, including setup and cleanup. We derive a quantitative consequence of the first paper's limiting inequality [9], then transfer it to native executions. This provides the lower-bound criterion used to exclude the concrete palettes and recursive schedules in Section 6. The first paper remarks that a common triangle wave fails its inequality; it does not contain an affine-motion theorem. The results below use only the $S = \varnothing$, $0 < \theta < 1/2$ instances of that inequality.

5.1 The crossing convention and the affine-time bound

Write λ for Lebesgue measure. For a path $q : [0, H] \to \mathbb{R}$ and a height p, let $\Lambda_q(p) = \{u \in [0, H] : q(u) = p\}$. Two visits $u < v$ are consecutive if no member of this *full* level set lies strictly between them. For an excision set S and a label $\beta : \mathbb{R} \to \mathbb{N}$, let $\kappa_q(p; S, \beta)$ count consecutive pairs with both endpoints outside S and with different labels. As in the first paper, this count is defined to be zero when $\Lambda_q(p)$ is infinite. In particular, removing S does not create new consecutive pairs.

A label is admissible off S if there is a finite set Z such that it is constant on every closed subinterval of $[0, H] \setminus (S \cup Z)$. The values at the finitely many breakpoints are unrestricted. All crossing integrals below are nonnegative Lebesgue integrals, with values in $[0, \infty]$.

Definition 5.1 (The restricted star condition)**.** For k paths $q_t : [0, H] \to \mathbb{R}$, write $\mathcal{S}(q, H)$ if, for every $0 < \theta < 1/2$, every finite union S of closed intervals, and every admissible label β whose classes off S have measure at most θ,

$$1 \le \sum_{t=1}^{k} \int_{\mathbb{R}} \kappa_{q_t}(p; S, \beta)\, dp + 2\theta + 2k\lambda(S \cap [0, H]). \tag{11}$$

A set $R_t \subseteq [0, H]$, open in $\mathbb{R}$, is an affine regular region for q_t if every point of R_t has an open neighborhood contained in R_t on which q_t is affine. The affine coefficients may depend on the neighborhood and the head.

Lean: `TransposeFollowup.FiniteAffineTimeBound`
`TransposeFollowup.AffineMotion.SatisfiesStar`

The original M^* conclusion implies $\mathcal{S}$; this restriction suffices for the obstruction below. The padding theorem retains the full original condition. Second countability gives each regular region a countable open affine cover, so countability is not an additional assumption. The cover sets need not be intervals or form a partition.

Theorem 5.2 (Quantitative affine-time obstruction)**.** *Let $H \geq 0$, let each q_t be 1-Lipschitz on $[0, H]$, and assume $\mathcal{S}(q, H)$. For any affine regular regions $R_1, \ldots, R_k$,*

$$\sum_{t=1}^{k} \lambda([0, H] \setminus R_t) \geq \frac{1}{2}. \tag{12}$$

The same conclusion holds when each R_t is open and q_t is affine on each connected component of R_t. If $k > 0$ and a common region R works for every head, then $\lambda([0, H] \setminus R) \geq 1/(2k)$.

Lean: `TransposeFollowup.OpenAffineTimeBound`
`TransposeFollowup.AffineMotion.star_half_le_locally_affine_omission`
`TransposeFollowup.AffineMotion.star_half_le_affine_component_omission`
`TransposeFollowup.AffineMotion.star_common_affine_omission`

The property (12), quantified over all affine regular regions, is named `AffineTimeRequired` in Lean.

Lean: `TransposeFollowup.NativeAffineMotion`
`TransposeFollowup.AffineMotion.AffineTimeRequired`

Outline of the formal proof. The labeling argument proves the following intermediate statement, which does not require $\mathcal{S}(q, H)$. For countable open affine covers $R_t = \bigcup_j U_{t,j}$, and any positive θ, ε, there is *one* admissible measurable label β with class measures at most θ such that

$$\sum_t \int \kappa_{q_t}(p; \varnothing, \beta)\, dp < 2 \sum_t \lambda([0, H] \setminus R_t) + \varepsilon. \tag{13}$$

For finitely many nonconstant affine branches, inverse visits at a common height are affine functions of that height. Bound retained crossing pairs by all ordered pairs of inverse branches and push their height measures to pairs of times. The resulting measures have explicit marginal bounds. The affine group is solvable and hence amenable: its Følner property provides finite sets of transformations with arbitrarily small relative boundary under any prescribed finite family [16]. Our formal proof constructs these sets directly, first choosing slopes and then translations. A fine grid and a common priority labeling of those sets make both label classes and the measure of disagreeing pairs small. Grid collisions and finite boundaries are charged separately. The finitely many constant-branch heights have measure zero; their time intervals are retained.

For a finite initial subfamily of the open cover, the complement in $[0, H]$ is compact. Deleting its visits can affect at most two consecutive pairs per deleted visit. A measurable visit-count majorant for a 1-Lipschitz path bounds the integrated charge by twice the complement's length. This uses the original full-level adjacency relation, without asserting that a clipped path has the same crossings. Finite-prefix complements decrease to the actual omitted set, so continuity of finite measure proves (13) for countably many branches. Substituting the common label into (11) with $S = \varnothing$, then sending θ, ε to zero, yields (12). A countable cover of any locally affine open region finishes the general case.

Lean: `TransposeFollowup.CountableAffinePathLabels`
`TransposeFollowup.AffineMotion.exists_countable_affine_path_cut_bound`

Lean: `TransposeFollowup.FiniteAffineLabels`
`TransposeFollowup.AffineMotion.exists_finite_affine_small_cut`

Lean: `TransposeFollowup.OmittedVisitBound`
`TransposeFollowup.AffineMotion.lintegral_levelCross_delete_le`

Lean: `TransposeFollowup.AffineFiniteBoundary`
`TransposeFollowup.FiniteBoundary.exists_affine_small_boundary`

Theorem 5.2 quantifies the time a candidate must leave outside its affine regions, even with countably many regions and slopes of either sign, including zero. The bound sums head-specific omitted measures; it does not require a common exceptional set for all heads. The Følner step connects the argument to the classical obstruction to expansion from amenability; see Lubotzky–Weiss [22]. The crossing-measure estimates and native time normalization are the additional steps needed here.

5.2 Native normalization and correct-input quantifiers

For the native results, let $d_i \to \infty$. After discarding a finite prefix, assume $d_i \geq 1$. Put $b_i = \max(1, \lceil \log_2 d_i \rceil)$, and $P_i = d_i^2 b_i$. Let M_i be native finite multitape machines with a fixed number k of work tapes and a fixed cell charge $G = \mathrm{cellBits}(M_i) \geq 1$. If Q_i is the number of finite-control states, assume

$$\frac{\lceil \log_2 Q_i \rceil}{P_i} \longrightarrow 0. \tag{14}$$

Let W_i be correct-input sets and $m_i \in \mathbb{N}$ satisfy, eventually,

$$2^{P_i} \leq 2^{m_i} |W_i|, \qquad m_i/P_i \longrightarrow 0. \tag{15}$$

Every member of W_i must be the raw row-major binary encoding of a $d_i \times d_i$ matrix, and the machine must actually halt with the raw encoding of its transpose. When we assume a fast execution, this halting must occur by cP_i ticks for one constant c. All these input and time conditions may hold only eventually in i. There is no condition on inputs outside W_i, and the machines themselves may vary with i.

Let $h_{i,t}^w(m) \in \mathbb{Z}$ be the position of work head t after m actual ticks on input w. The native trajectory is frozen after halting. Physical time and space are measured in units of P_i:

$$f_{i,t}^w(s) = \frac{\widetilde{h}_{i,t}^w(P_i s)}{P_i}, \qquad \widehat{f}_{i,t}^w(s) = \frac{h_{i,t}^w(\lfloor P_i s \rfloor)}{P_i}, \tag{16}$$

where $\widetilde{h}$ is any 1-Lipschitz extension agreeing at integer ticks. Such an extension exists, and the two expressions differ by at most $1/P_i$ for $s \geq 0$.

The block normalization in [9] is different at finite size. Put $r_i = \lceil d_i b_i / G \rceil$. Its actual block trajectory is

$$B_{i,t}^w(u) = \frac{1}{d_i} \left\lfloor \frac{h_{i,t}^w(r_i \lfloor d_i u \rfloor)}{r_i} \right\rfloor,$$

where the outer floor is the integer block index, also for negative head positions. The proved comparison is

$$\left| f_{i,t}^w(u/G) - B_{i,t}^w(u)/G \right| \leq \frac{2(u+1)}{d_i} \quad (u \geq 0). \tag{17}$$

Thus a block limit q_t corresponds to the physical limit $f_t(s) = q_t(Gs)/G$, although the finite trajectories differ.

The next theorem assembles the cited native-limit and scaling declarations; it is not a single exported Lean statement.

Theorem 5.3 (Physical units and native limiting paths)**.** *Under* (14)–(15) *and fast correct halting, there are representatives $w_i \in W_i$ eventually and a subsequence whose block trajectories converge uniformly on $[0, cG + 1]$. Every uniform subsequential limit of these representatives is* 1*-Lipschitz and satisfies* (12)*. For the corresponding physical limits on $[0, c + 1/G]$, any affine regular regions satisfy*

$$\sum_{t=1}^{k} \lambda([0, c + 1/G] \setminus R_t) \geq \frac{1}{2G}. \tag{18}$$

When $k > 0$, a common open region on each of whose connected components every head is affine omits at least $1/(2kG)$ physical time. The assertion includes a fixed uniform machine as a special case.

Lean: `TransposeFollowup.NativeAffineMotion`
`TransposeFollowup.AffineMotion.popular_affine_time_nonuniform`
`TransposeFollowup.AffineMotion.popular_affine_time_robust`
`TransposeFollowup.AffineMotion.AffineTimeRequired.physical_omission`

Lean: `TransposeFollowup.AffineTimeScaling`
`TransposeFollowup.AffineMotion.star_physical_common_affine_omission`

Lean: `TransposeFollowup.NativePhysicalLimits`
`TransposeFollowup.RegularMotion.native_sampled_physical_limit`

Lean: `TransposeFollowup.CanonicalTimeNormalization`
`TransposeFollowup.RegularMotion.canonical_physical_error`

Imported Lean: `Transpose.Tape.MStarRobust`
`Transpose.Tape.Impl.theoremMStar_level_nonuniform`

Outline of the formal proof. The popular-trace theorem of [9] supplies actual trace classes $F_i \subseteq W_i$ with representatives w_i, common halting times, and integer deficits D_i satisfying $D_i \leq m_i + C_0 d_i b_i$ eventually, for a fixed trace-count constant C_0. Consequently $D_i/P_i \to 0$. Its native compactness and M^* theorem applies to these classes; Theorem 5.2 then applies to every eligible limit. The estimate (17) transfers uniform convergence to physical paths, including all phases of execution and the frozen halting tail. Finally the change of variables $u = Gs$ scales omitted measure by G. For the common-region clause, retain the original M^* theorem's star condition and apply the cited common-region scaling theorem. Its exported hypothesis is affinity on each connected component. For an open subset of $\mathbb{R}$ this is equivalent to local affinity: components are open intervals, and affine formulas agree on overlapping open neighborhoods, hence throughout each connected component.

This theorem derives the input sequence and limiting obstruction from native correctness. It lets a lower-bound argument analyze physical motion without assuming a favorable trace class or conflating block time with execution time. The toll is only $1/(2G)$ in total head-time, on the scale of half a scan of a nominal P/G-cell storage footprint. It is a bound on time outside affine regions, not on distance travelled, and does not itself give a superlinear time lower bound.

Definition 5.4 (Bounded affine interval cover)**.** A function $v : [0, H] \to \mathbb{R}$ has an affine interval cover of size J if there are J endpoint pairs $\ell_j, r_j \in [0, H]$ such that their closed intervals cover $[0, H]$ and v is affine on each $[\ell_j, r_j]$. Intervals may overlap or collapse; a pair with $\ell_j > r_j$ contributes an empty interval. No continuity or Lipschitz assumption is imposed on v.

Lean: `TransposeFollowup.BoundedAffineCovers`
`TransposeFollowup.AffineMotion.HasAffineIntervalCover`

Lemma 5.5 (Pointwise closure)**.** *If v_i eventually have affine interval covers of a fixed finite size J and $v_i(s) \to v(s)$ at every $s \in [0, H]$, then v is locally affine on an open subset of $[0, H]$ whose complement has measure zero.*

Lean: `TransposeFollowup.BoundedAffineCovers`
`TransposeFollowup.AffineMotion.local_affine_ae_of_eventual_bounded_covers`

Outline of the formal proof. Pass to a subsequence on which all $2J$ endpoints converge. Their limiting closed intervals still cover $[0, H]$. Away from the finitely many limiting endpoints and the horizon endpoints, every point has a neighborhood lying inside one interval for all sufficiently large indices. On a smaller fixed neighborhood the functions are affine. Their coefficients converge because the values at two distinct fixed points converge. The pointwise limit is affine there.

This closure lemma is the step that makes pointwise approximation sufficient for the next native obstruction; no slope bound or uniform convergence of the approximants is needed.

Theorem 5.6 (Pointwise approximation obstruction)**.** *The native fast-correct hypotheses above are incompatible with the following property. For every strictly increasing $\phi : \mathbb{N} \to \mathbb{N}$ and every input sequence (w_i) eventually satisfying $w_i \in W_i$, there exist a strictly increasing ψ, a finite J, and functions $v_{i,t}$ such that, with $H = c + 1/G$:*

1. *eventually every $v_{i,t}$ has an affine interval cover of size J on $[0, H]$;*
2. *for every head t and every fixed $s \in [0, H]$,*

$$|\widehat{f}^{\,w_{\phi(\psi(i))}}_{\phi(\psi(i)),t}(s) - v_{i,t}(s)| \longrightarrow 0.$$

Here J, the further subsequence, and the approximants may depend on the chosen ϕ and the entire input sequence. Only J must be fixed along that further subsequence.

Lean: `TransposeFollowup.PhysicalAffineApproximationObstruction`
`TransposeFollowup.AffineMotion.no_subsequential_physical_pointwise_affine_fast_approximation`

Outline of the formal proof. Choose the popular representatives and a convergent subsequence supplied by Theorem 5.3, then apply the asserted approximation property to precisely those choices. The approximants converge pointwise to the same physical limits. Lemma 5.5 makes every head locally affine outside a null set, contradicting (18).

Theorem 5.6 reduces an exclusion proof to bounded-cover approximation of every work head over the whole physical horizon. Error rates may vary with time, but convergence is required at *every* fixed time. Regularity of a selected subroutine alone does not suffice.

6 Concrete palettes and contracting recursion

We apply the motion obstruction to bounded words in explicit operation palettes and to contracting trees of execution stages. The palette proofs supply the native implementations and their geometry. The recursive result requires the stated regularity condition on actual stages.

6.1 The declared palettes

All indices below start at zero. Set $W = db$ and write the bank as $x_{r,c,\ell}$, with $0 \le r, c < d$ and $0 \le \ell < b$, at flattened position $rW + cb + \ell$. A displayed formula specifies the output y in terms of the input x to *that operation*; a word applies its operations successively. The following four symbol permutations are shared by the integer and ternary palettes. They preserve the order of the b lanes inside each entry.

Operation	Exact output coordinate
R	$y_{r,c,\ell} = x_{d-1-r,c,\ell}$
C	$y_{r,c,\ell} = x_{r,d-1-c,\ell}$
E	$y_{r,c,\ell} = x_{r,(c+r) \bmod d,\ell}$
e	$y_{r,c,\ell} = x_{r,(c-r) \bmod d,\ell}$

Residues modulo d are represented in $\{0, \ldots, d-1\}$. The corresponding declarations are `rows`, `entries`, and the two choices of `rotate`.

Lean: `TransposeFollowup.MatrixPermutationPalette`
`TransposeFollowup.MatrixPermutationPalette.Operation`
`TransposeFollowup.MatrixPermutationPalette.value`

Lean: `TransposeFollowup.RawEntryRotation`
`TransposeFollowup.RawEntryRotation.index_entry`

Lean: `TransposeFollowup.RawEntryReversal`
`TransposeFollowup.RawEntryReversal.index_entry`

For the *integer palette*, interpret each complete raw row as a big-endian integer X_r modulo 2^W, and the complete bank as a big-endian integer X modulo 2^P. In addition to R, C, E, e, permit exactly the following operations:

Operation	Output rule
Row sum	$Y_r = \sum_{a=0}^{r} X_a \pmod{2^W}$
Row difference	$Y_0 = X_0$, $Y_r = X_r - X_{r-1} \pmod{2^W}$ for $r > 0$
Row XOR prefix	$y_{r,c,\ell} = \bigoplus_{a=0}^{r} x_{a,c,\ell}$
Row XOR difference	$y_{0,c,\ell} = x_{0,c,\ell}$; $y_{r,c,\ell} = x_{r,c,\ell} \oplus x_{r-1,c,\ell}$ for $r > 0$
Increment/decrement	$Y = X + 1 \pmod{2^P}$ or $Y = X - 1 \pmod{2^P}$

Carries in row arithmetic can cross entry boundaries but not row boundaries. A difference uses the preceding *original* row; a prefix uses its previous accumulated output. Increment and decrement act on the whole bank.

Lean: `TransposeFollowup.IntegerMatrixPalette`
`TransposeFollowup.IntegerMatrixPalette.Operation`
`TransposeFollowup.IntegerMatrixPalette.value`

Lean: `TransposeFollowup.RawRowArithmeticData`
`TransposeFollowup.RawRowArithmetic.output_value`

Lean: `TransposeFollowup.RawRowXor`
`TransposeFollowup.RawRowXor.bitValue_first`
`TransposeFollowup.RawRowXor.bitValue_next`

Lean: `TransposeFollowup.RawUnitArithmeticData`
`TransposeFollowup.RawUnitArithmetic.output_value`

For the *ternary palette*, the same coordinates hold values in $\mathbb{F}_3$. There are b independent trit lanes per entry; these are not integers represented in base three. Along with R, C, E, e, the complete additional palette is:

Operation	Exact output coordinate in $\mathbb{F}_3$
P	$y_{r,c,\ell} = \sum_{a=0}^{r} x_{a,c,\ell}$
p	$y_{r,c,\ell} = x_{r,c,\ell} - x_{r-1,c,\ell}$, with $x_{-1,c,\ell} = 0$
Q	$y_{r,c,\ell} = \sum_{a=0}^{c} x_{r,a,\ell}$
q	$y_{r,c,\ell} = x_{r,c,\ell} - x_{r,c-1,\ell}$, with $x_{r,-1,\ell} = 0$
D	$y_{r,c,\ell} = (-1)^{\mathrm{popcount}(r \& c)} x_{r,c,\ell}$
S, s	$y_{a,\ell} = x_{(a+1) \bmod d^2,\ell}$ or $y_{a,\ell} = x_{(a-1) \bmod d^2,\ell}$

In the last line $a = rd + c$ is the flattened *entry* index. In the phase operation & is bitwise AND and popcount counts its set bits. Binary inputs enter as trits $0, 1$, and intermediate trits are retained exactly. Final emission applies $\pi(0) = 0, \pi(1) = 1, \pi(2) = 0$; the theorem therefore also excludes words whose final bank is already binary.

Lean: `TransposeFollowup.TernaryMatrixPalette`
`TransposeFollowup.TernaryMatrixPalette.Operation`
`TransposeFollowup.TernaryMatrixPalette.value`

Lean: `TransposeFollowup.RawTernaryVertical`
`TransposeFollowup.RawTernaryVertical.value_entry`

Lean: `TransposeFollowup.RawTernaryMatrixHorizontal`
`TransposeFollowup.RawTernaryMatrixHorizontal.value_entry`

Lean: `TransposeFollowup.TernaryPhase`
`TransposeFollowup.TernaryPhase.value_encoded_entry`

Lean: `TransposeFollowup.RawEntryShift`
`TransposeFollowup.RawEntryShift.left_value_entry`
`TransposeFollowup.RawEntryShift.right_value_entry`

Lean: `TransposeFollowup.RawSymbolWord`
`TransposeFollowup.RawSymbolWord.output`

The *mixed binary palette* acts on $x_0, \dots, x_{P-1} \in \mathbb{F}_2$. A masked stride operation chooses a public mask $\mu : \mathbb{N} \to \{0, 1\}$, a stride $1 \le S \le P$, a direction, and prefix or difference mode. In the forward direction it computes in increasing index order

$$y_j = x_j \oplus (\mu(j) z_j), \qquad z_j = \begin{cases} 0, & j < S, \\ y_{j-S}, & j \ge S \text{ in prefix mode}, \\ x_{j-S}, & j \ge S \text{ in difference mode}. \end{cases}$$

Backward mode uses $j + S$ in place of $j - S$, computes in decreasing order, and uses zero when $j + S \ge P$. The remaining operations are full reversal $y_j = x_{P-1-j}$ and rotation $y_j = x_{(j+R) \bmod P}$ with $0 \le R \le P$. These are all the operations in this palette. The masks, strides, offsets, and operation words may depend arbitrarily on size, but not on the input.

Lean: `TransposeFollowup.BinaryBankOperation`
`TransposeFollowup.BinaryBankOperation.Operation`
`TransposeFollowup.BinaryBankOperation.Valid`
`TransposeFollowup.BinaryBankOperation.value`

Lean: `TransposeFollowup.MaskedStrideScan`
`TransposeFollowup.MaskedStrideScan.buffer`
`TransposeFollowup.MaskedStrideScan.output`

Lean: `TransposeFollowup.BackwardMaskedStride`
`TransposeFollowup.BackwardMaskedStride.value_recurrence`

Theorem 6.1 (Bounded-word palette exclusions)**.** *Let $d_i \to \infty$ and use the canonical b_i, P_i above. Fix any finite K. For each of the three declared palettes, there is no sequence of words ω_i of length at most K eventually whose output equals raw transpose eventually on sets W_i of correctly encoded inputs satisfying (15). For mixed binary words, every stride and rotation must satisfy the stated bounds. Public choices may vary arbitrarily with i; there is no uniform-generation premise.*

Lean: `TransposeFollowup.MatrixPaletteObstructions`
`TransposeFollowup.IntegerMatrixPalette.no_high_entropy_transpose`
`TransposeFollowup.TernaryMatrixPalette.no_high_entropy_transpose`

Lean: `TransposeFollowup.RawMixedBinaryWordObstruction`
`TransposeFollowup.RawMixedBinaryWord.no_high_entropy_transpose`

Outline of the formal proof. Each word is compiled to one actual native execution. The input is loaded once, successive primitives run on the states produced by the preceding primitives, and the output is emitted once. Each primitive's contract preserves the bank semantics, clears its scratch bank, and accounts for rewinds and head restoration.

For the integer and ternary palettes, every primitive costs at most $24P$ and the input/output shell costs $7P$. The executor has two work tapes, five work symbols, $G = 3$, and exactly $200(T + 1)$ control states when its paid time is T. Thus $T \le (7 + 24K)P$ and its state-bit charge is $o(P)$. Put $J_0 = 81^3 \cdot 9$. At arbitrary prefix times and from arbitrary actual stage states, every primitive has an affine-cover approximation with at most J_0 pieces and unnormalized error at most $32(db + 1)$. The complete word, including the shell and frozen tail, therefore has at most $27J_0^K$ pieces and normalized error at most

$$\frac{32(db + 1)K + 4}{P} \longrightarrow 0.$$

The mixed binary executor has the same two tapes and five-symbol alphabet, with $T \le (7 + 27K)P$ and $4(T + 1)$ control states. Strides need not be small relative to P. The geometry proof therefore selects subsequences according to their normalized sizes: a vanishing stride has a vanishing-error macroscopic scan approximation, while a stride bounded below as a fraction of P permits only boundedly many groups. The resulting further subsequence has bounded affine covers. Masks change the data rule without changing the paid head schedule. The input/output phases are then added with their proved error. All three executors meet Theorem 5.6, which contradicts the assumed correctness sets.

Lean: `TransposeFollowup.RawSymbolWord`
`TransposeFollowup.RawSymbolWord.native_correct`
`TransposeFollowup.RawSymbolWord.resources`
`TransposeFollowup.RawSymbolWord.ticks_le`

Lean: `TransposeFollowup.RawSymbolWordApproximation`
`TransposeFollowup.RawSymbolWord.scaled_approximation`
`TransposeFollowup.RawSymbolWord.state_bits_vanish`

Lean: `TransposeFollowup.IntegerMatrixGeometry`
`TransposeFollowup.IntegerMatrixPalette.approximation_cover`
`TransposeFollowup.IntegerMatrixPalette.approximation_error`

Lean: `TransposeFollowup.TernaryMatrixGeometry`
`TransposeFollowup.TernaryMatrixPalette.approximation_cover`
`TransposeFollowup.TernaryMatrixPalette.approximation_error`

Lean: `TransposeFollowup.RawMixedBinaryWord`
`TransposeFollowup.RawMixedBinaryWord.fixed_resources`

Lean: `TransposeFollowup.RawMixedBinaryWordResources`
`TransposeFollowup.RawMixedBinaryWord.bounded_ticks`
`TransposeFollowup.RawMixedBinaryWord.state_bits_vanish`

Lean: `TransposeFollowup.RawMixedBinaryWordRegularity`
`TransposeFollowup.RawMixedBinaryWord.full_affine_approximation_subsequence`

Theorem 6.1 rules out these bounded-word construction families without a caller-supplied geometry assumption. A surviving word construction must leave this class, for example by using unboundedly many operations, input-dependent selection, or another primitive. The asymptotic exclusion is at $b = b(d)$; implementations of the primitives at other widths do not by themselves extend it to those widths.

6.2 Paid contracting stage clocks

A contracting recursion may have unboundedly many stages. To analyze it, we assign every actual tick to a stage and bound the total duration of deep stages. A separate regularity condition will control each fixed stage's physical motion.

Definition 6.2 (Contracting stage clock)**.** Fix natural numbers B, J and real parameters $0 \le \rho < 1$, $a \ge 0$, and C. For an execution of T ticks at input size $P > 0$, a clock consists of nonnegative volumes $V(w)$ on addresses $w \in \{0, \dots, B-1\}^*$, a label $\ell : \{0, \dots, T-1\} \to \{0, \dots, B-1\}^* \times \{0, \dots, J-1\}$, and nonnegative integer endpoints $A(w, j) \le E(w, j) \le T$, such that

$$\sum_{b=0}^{B-1} V(wb) \le \rho V(w), \qquad V(\varnothing) \le CP, \tag{19}$$

$$A(w, j) \le t < E(w, j) \iff \ell(t) = (w, j), \qquad (0 \le t < T), \tag{20}$$

$$\sum_{j=0}^{J-1} (E(w, j) - A(w, j)) \le aV(w). \tag{21}$$

The constants are fixed across the family, but the clock may depend on the execution. Empty stages are allowed.

Lean: `TransposeFollowup.ContractingStageClock`
`TransposeFollowup.RegularMotion.ContractingStageClock`

Equation (20) partitions the global integer timeline. Each own-stage is contiguous; the work of a node together with its descendants need not be contiguous or nested. The clock thus covers more schedules than depth-first recursion. Its volume $V(w)$ is an accounting quantity, not an assumption of independently accessible memory.

Clock existence alone does not restrict linear-time executions when the family constants may be chosen: if $T \le cP$, take $B = J = 1$, $\rho = 0$, $a = 1$, $C = c$, $V(\varnothing) = cP$, and one root stage $[0, T)$; give every other node zero volume and an empty stage. The substantive additional hypothesis below is regularity of the stages marked good.

Lemma 6.3 (Charged depth and exceptional work)**.** *For any such clock and $h \in \mathbb{N}$,*

$$\#\{t < T : |\ell(t)_1| \ge h\} \le \frac{aV(\varnothing)\rho^h}{1-\rho}, \qquad T \le \frac{aCP}{1-\rho}. \tag{22}$$

Mark each stage good or exceptional, and let E_{bad} count all ticks owned by exceptional stages. The part of $[0, T/P)$ not covered by good stages of depth less than h has measure at most

$$\frac{E_{\mathrm{bad}}}{P} + \frac{aC\rho^h}{1-\rho}. \tag{23}$$

If exceptional ticks owned by each node w number at most $\eta V(w)$, where $\eta \ge 0$, then

$$E_{\mathrm{bad}}/P \le \eta C/(1-\rho). \tag{24}$$

Lean: `TransposeFollowup.StageClockAccounting`
`TransposeFollowup.RegularMotion.stage_clock_deep_work_bound`
`TransposeFollowup.RegularMotion.stage_clock_total_time_bound`

Lean: `TransposeFollowup.ExceptionalStageClock`
`TransposeFollowup.RegularMotion.regular_shallow_stage_omission`

`TransposeFollowup.RegularMotion.exceptionalTicks_node_budget`

Outline of the formal proof. The total volume at depth r is at most $\rho^r V(\varnothing)$. Ownership identifies the number of ticks at one node with the sum of its stage lengths. Summing its own-work budget over depths at least h gives the geometric tail in (22). The finite integer intervals in (20), divided by P, have measure exactly their numbers of ticks divided by P. Their disjointness is derived from the ownership equivalence. A tick outside the shallow good stages is either exceptional or deep, giving (23). Charging exceptional ticks instead by $\eta V(w)$ gives (24), with each tick counted once.

The accounting lemma supplies the recursive obstruction's linear time bound and controls the work lost when only finitely many depths are retained. Its per-node estimate will convert local exceptional-work budgets into a global lower bound.

6.3 Actual-stage regularity and the exceptional-time toll

Definition 6.4 (Regular sequences of stage occurrences)**.** Let V_i, A_i, L_i be the volume, actual start tick, and duration of one fixed stage across a sequence of executions, and let h_i be one actual integer head path. These occurrences are regular if, for every strictly increasing ϕ such that $L_{\phi(i)} > 0$ for all i and $V_{\phi(i)} \to \infty$, there are a further strictly increasing ψ, a finite K, and a function v with an affine interval cover of size K on $[0, a]$ for which

$$\frac{h_{\phi(\psi(i))}\Big(A_{\phi(\psi(i))} + \min\{\lfloor V_{\phi(\psi(i))} s \rfloor, L_{\phi(\psi(i))}\}\Big) - h_{\phi(\psi(i))}(A_{\phi(\psi(i))})}{V_{\phi(\psi(i))}} \longrightarrow v(s) \tag{25}$$

uniformly for $0 \le s \le a$. A sequence of flagged clocks is regular for a head if this condition holds for every fixed address and stage, using its true duration when the stage is good and duration zero otherwise.

Lean: `TransposeFollowup.ActualStageRegularity`
`TransposeFollowup.RegularMotion.HasRegularStageSubsequences`

Lean: `TransposeFollowup.ExceptionalRecursiveLimits`
`TransposeFollowup.RegularMotion.RegularFlaggedClockHead`

The profile uses global ticks and freezes at the stage's actual end. Its cover size and limit may depend on the fixed stage, head, and subsequences; there is no common piece bound across stages. Missing, exceptional, and bounded-volume occurrences impose no diverging positive-duration obligation.

Call a native family *flagged-recursive with allowance* (η, ε_i) if there exists N such that, for every strictly increasing ϕ with $\phi(i) \ge N$, every sequence $w_i \in W_{\phi(i)}$, and every sequence of their actual halting times T_i with $P_{\phi(i)} > 0$, there exist clocks and stage flags satisfying

$$E_{\mathrm{bad},i}/P_{\phi(i)} \le \eta + \varepsilon_{\phi(i)},$$

and every work head is regular in Definition 6.4. The sequence $\varepsilon_i \to 0$ is fixed for the family. Clocks and flags may depend on the execution sequence; no uniform algorithm for choosing them is required.

Lean: `TransposeFollowup.NativeExceptionalRecursion`
`TransposeFollowup.RegularMotion.HasFlaggedRecursiveExecutions`

Theorem 6.5 (Exceptional-time toll for contracting recursion)**.** *Assume the native resources, state-bit bound, and high-entropy correct sets of* (14)–(15)*. Fix the clock parameters B, J, ρ, a, C as above. Suppose every correct input eventually has an actual halting transpose execution admitting*

a contracting stage clock, and the family is flagged-recursive with allowance (η, ε_i)*, where* $\varepsilon_i \to 0$*. Then*

$$\frac{1}{2G} \le k\eta, \qquad \eta \ge \frac{1}{2kG}. \tag{26}$$

In particular, no such correct family can have every stage regular and zero exceptional work. The assertion holds for a fixed uniform machine as well. If a linear-time bound is already assumed, the first inequality follows without separately postulating clock existence for each correct execution.

Lean: `TransposeFollowup.NativeExceptionalRecursion`
`TransposeFollowup.RegularMotion.native_flagged_recursion_budget`
`TransposeFollowup.RegularMotion.native_fast_flagged_recursion_budget`
`TransposeFollowup.RegularMotion.uniform_flagged_recursion_budget`

Lean: `TransposeFollowup.NativeRegularRecursion`
`TransposeFollowup.RegularMotion.no_native_regular_recursion`
`TransposeFollowup.RegularMotion.no_uniform_regular_recursion`

Outline of the formal proof. The clock gives $T_i \le aCP_i/(1-\rho)$. Apply the popular-trace and native limit theorem, discard a finite prefix, and invoke the family-level clock hypothesis on the resulting representatives.

A diagonal subsequence supplies limits of the normalized stage endpoints, volumes, and flags at every fixed node. If a limiting good-stage interval has positive length, its volume is bounded below by a positive multiple of P_i, by the own-work budget. Hence its volume tends to infinity and its duration is eventually positive. Definition 6.4 then applies. Transferring (25) to the global physical limit makes that limit locally affine on the interior of the stage apart from finitely many breakpoints. Zero-length limiting stages contribute no time.

At each fixed depth there are finitely many stages. Bound (23) passes to their limiting intervals; sending the depth to infinity leaves omitted measure at most η. The countable union of the locally affine stage regions therefore gives, for each head, an open regular region missing at most η physical time. The frozen post-halting tail is constant and is included in this region up to its endpoint. Thus (18) gives $1/(2G) \le k\eta$. This also forces $k > 0$, yielding the second inequality. All-good flags and $\eta = 0$ prove the regular-recursion exclusion.

Lean: `TransposeFollowup.ExceptionalRecursiveLimits`
`TransposeFollowup.RegularMotion.flagged_clock_limit_locally_affine`

Lean: `TransposeFollowup.ActualStageRegularity`
`TransposeFollowup.RegularMotion.positive_stage_affine_of_regular_occurrences`

Lean: `TransposeFollowup.NativeRecursiveLimit`
`TransposeFollowup.RegularMotion.native_flagged_limit_budget`

Corollary 6.6 (Per-node exceptional budget)**.** *Under the same native correctness and clock assumptions, replace the global exceptional allowance by the following family-level condition: in every eligible execution sequence one can choose clocks and flags with the same regularity and with at most* $\eta V_i(w)$ *exceptional ticks owned by each node* w*, for a fixed* $\eta \ge 0$*. Then*

$$\frac{1}{2G} \le k\frac{\eta C}{1-\rho}, \qquad \frac{\eta C}{1-\rho} \ge \frac{1}{2kG}.$$

Lean: `TransposeFollowup.NativeExceptionalRecursion`
`TransposeFollowup.RegularMotion.HasNodeBudgetRecursiveExecutions`
`TransposeFollowup.RegularMotion.native_node_budget_recursion_bound`

Outline of the formal proof. The exact node charge in (24) produces a global allowance $\eta C/(1-\rho)$ with error sequence identically zero. Apply Theorem 6.5.

Corollary 6.6 gives a necessary lower bound on per-node exceptional work. Together with Theorem 6.5, it excludes variants of the geometric recursion in [9, Section 13] whose actual stages all have the regularity in Definition 6.4. It does not exclude the proposed non-affine Möbius choreography: a non-affine Möbius map is affine on no nonempty open interval of its domain, so a stage limit agreeing with such a map on an interval of positive length has no finite affine cover. The construction still has to realize its intended profiles on actual tapes. The exceptional allowance must be at least $1/(2kG)$, a constant physical-time toll corresponding asymptotically to $P/(2kG)$ ticks. This toll does not grow with the recursion depth.

These exclusions concern $O(P)$ transposition at canonical logarithmic entry width. Paid padding, proved next, shows that the first paper's necessary M^* condition can be supplied at additive linear cost to any linear-time payload. A construction must therefore both escape the excluded motion classes and perform the required computation; a general lower bound must identify an obligation that padding cannot satisfy independently. Neither these restricted exclusions nor padding decides the unrestricted transposition question.

7 Fully paid padding and the limits of the motion test

A native-machine prefix of linear cost enforces the original cut inequality through any payload computation. For uniformly linear-time payload families, the resulting runs also satisfy the full limiting conclusions. This supplies a data-free realization of those conditions; coupling the motion to a transposition computation remains open.

7.1 The exact geometric contract

We recall the discrete condition from [9]. Let k unit-step work-head paths $h_p : \mathbb{N} \to \mathbb{Z}$ run for T steps. At dimension d, take $r = \lceil db(d)/G \rceil$ and split time into $N = \lceil T/r \rceil$ slots. A block is a length-r interval of tape cells. The original *pair residency* of a slot is the union, over every step of that slot before T, of the current block and its previous block under the original last-change convention. A chord joins resident slots $x < z$ only when $x + 1 < z$ and the block is absent from every intervening slot: it connects the last slot of one residency visit to the first slot of the next separated visit. Adjacent resident slots do not form chords. The theorem uses this original residency, not an arbitrary collection of convenient edges.

Give the slots labels by $\lambda : \mathbb{N} \to \mathbb{N}$, with at most m slots of any one label among $0, \dots, N-1$. Let B_λ be the number of adjacent label changes and $\chi_p(\lambda)$ the number of residency chords of head p whose endpoints have different labels. The *zero-slack cut inequality* is

$$(d - B_\lambda)_+ (d - 2m)_+ b(d) \le rG \left(3kB_\lambda + \sum_{p<k} \chi_p(\lambda) \right) \tag{27}$$

for every such labelling and m, where $x_+ = \max(x, 0)$. It is precisely `CutIneq d G r T 0 0 h`; the two zero parameters are the entropy-deficit and boundary-information slack, not a restriction on the payload's input distribution.

Imported Lean: `Transpose.Tape.Defs`
`Transpose.Tape.pairRes`
`Transpose.Tape.labelChanges`
`Transpose.Tape.crossChords`

Imported Lean: `Transpose.Tape.DefsM`
`Transpose.Tape.CutIneq`

For a linear time bound $T_i \le CP(d_i)$, freeze each path at T_i and put

$$p_{i,t}(u) = \frac{1}{d_i}\left\lfloor \frac{h_{i,t}(\min(\lfloor ud_i \rfloor r_i, T_i))}{r_i} \right\rfloor, \qquad 0 \le u \le c_0 := CG + 1.$$

These are the original block trajectories. The full limiting conclusion means that some subsequence converges uniformly, and *every* uniformly convergent subsequence has 1-Lipschitz limit paths satisfying the original M^* inequality for every $\theta \in [0, \infty]$, every finite union S of closed intervals, and every piecewise-constant-off-S labelling whose off-S classes each have measure at most θ. The inequality is (11) with this full range of θ. In addition, their total integrated level excess is at least one, the sum of their range measures plus one is at most kc_0, and at least one limit path is neither monotone nor antitone.

Lean: `TransposeFollowup.NativeRunLimits`
`TransposeFollowup.NativeRunLimits.Conclusion`
`TransposeFollowup.NativeRunLimits.of_cut`

7.2 The wrapper theorem

Theorem 7.1 (Universal fully paid padding)**.** *There are absolute integers $A, H, K_0, E \ge 0$ such that for every native machine M there is a single native machine M' with*

$$\begin{aligned} \#\text{states}(M') &= H + \#\text{states}(M), \\ k(M') &= K_0 + (k(M) + E)\max(1, \text{size}(g(M))), \\ \text{cellBits}(M') &= 3. \end{aligned}$$

Here $g(M) + 1$ is the work-alphabet size and size *is the binary length of a natural number, with* $\text{size}(0) = 0$*. The following hold for this same M'.*

(a) For every $d \ge 1$, every word w of length $P(d)$, every output v and every t for which M computes v on w within t steps, there is an actual halting time $T \le AP(d) + t$ of M' on w, with output exactly v, whose entire execution satisfies (27) with $G = 3$ and $k = k(M')$.

(b) For every sequence $d_i \to \infty$ of positive dimensions, words w_i of length $P(d_i)$, outputs v_i, and $a \in \mathbb{N}$ such that M computes v_i on w_i within $aP(d_i)$ steps, there are actual halting times $T_i \le (A + a)P(d_i)$ for M' with those same outputs and the cut inequality. The full limiting conclusion above holds with $C = A + a$.

The constants precede the choice of M, and M' precedes every choice of dimension, input sequence, output and time coefficient.

Lean: `TransposeFollowup.UnconditionalPaidPadding`
`TransposeFollowup.PaidPadding.exists_uniform_paid_padding`

What is paid. The machine starts with blank work tapes. It derives the required dimension from the input length, generates its finite schedule, builds the clocks used to execute it, performs the scheduled excursions, clears the prefix workspace, resets the input head, and hands off to a faithful simulation of M. All transitions are included in $AP + t$. The implementation uses five work symbols; the headline contract states their three-bit cell charge. Parallel bit planes encode the source alphabet using additional tapes, which is why the displayed tape count depends on the source alphabet while the additive time coefficient can be chosen before M.

The tape count includes $E \max(1, \mathrm{size}(g(M)))$ idle tapes: they remain blank with their heads at cell zero throughout the run. In the construction, $K_0 = 53 + q$, where q is the page count of the fixed symmetric rational family. The extra tapes are chosen to ensure $4q \le 3k(M')\eta$, where $\eta > 0$ is the finite graphs' vertex-expansion constant. This is the head-count hypothesis of `PaidPayload.cut_after_prefix`: the term $3k(M')rGB_\lambda$ in (27) absorbs the expanding edges lost at label changes. Thus this finite cut guarantee uses the coarse per-tape memory-boundary charge. An entry-cell charge would count at most one cell per boundary on each idle tape, rather than three whole blocks. The theorem does not establish the corresponding inequality with that finer charge.

Lean: `TransposeFollowup.StationaryTapesRun`
`TransposeFollowup.StationaryTapes.observations`

Lean: `TransposeFollowup.BitPlaneMachine`
`TransposeFollowup.BitPlaneMachine.fixed_resources`

Lean: `TransposeFollowup.BitPlaneRun`
`TransposeFollowup.BitPlaneMachine.run_embed`

Lean: `TransposeFollowup.PaidPaddingLimits`
`TransposeFollowup.PaidPadding.fixed_resources`
`TransposeFollowup.PaidPayload.cut_after_prefix`

The idle paths contribute zero to the integrated level-crossing and excess terms and are both monotone and antitone. Removing them therefore preserves the $S = \varnothing$ instances of M^*, the excess lower bound of one, and the existence of a nonmonotone, nonantitone path. These limiting conclusions already concern the remaining heads; they do not rely on the idle tapes' memory-boundary charge.

The constants are existential; no numerical values are given here. Once the rational family and its parameters are fixed, the same transition table handles all positive canonical dimensions.

Outline of the formal proof. The proof has four parts.

Analytic existence. A fixed finite rational family of increasing partial Möbius maps expands measurable subsets of the unit interval. The development follows the continuous-expander strategy of Bourgain–Yehudayoff [7]: rational free generators near the identity, flattening and a restricted spectral gap, then the passage from a gap to expansion. The formal chain uses the pinned scalar-projection result [26] at its additive-combinatorial interface; its local concentration hypotheses are discharged in our analytic development. No spectral-gap or expansion hypothesis is left in the wrapper theorem.

Finite graphs and excursions. Divide the interval into d cells. An edge records an overlap of a map image with a target cell. Symmetrization gives a graph on $2d$ ordered vertices with a fixed positive vertex-expansion constant. The estimates hold at every positive dimension. Monotonicity gives a fixed finite page decomposition of these edges. A page is implemented by one head's excursions, with a constant stretch chosen so that expanding edges force enough distinct crossing chords in every slot labelling. A fixed number of idle tapes supplies the head-count inequality described above.

Actual generation and execution. Rational comparisons determine the overlap edges and the corresponding finite operation stream. The generator and its consumer are compiled to finite transition tables. The row-clock and excursion proofs identify their actual head positions at the actual transition times, including fetching and setup delays. Their aggregate cost, including generation, is $O(P)$. Cleanup proves blank tape contents and zero head positions, and resetting the input head supplies the exact entry contract for the payload. The five-symbol bit-plane simulation preserves one source transition per payload transition. Sequential composition therefore preserves the output and gives the stated additive time bound.

Extending the horizon and taking limits. The prefix excursions witness the cut inequality through the payload's final halting time, using the full composed machine's head count and paths. Finally `NativeRunLimits.of_cut` applies the original walk theorem with zero slack, giving existence of limits and the full assertion for every eligible limit. The construction parameters are chosen before the payload machine, as required by the outer quantifiers.

7.3 The analytic interface and its attribution

The following analytic lemma removes the expansion hypothesis from the native construction. Continuous-expander existence and the proof strategy are due to Bourgain–Yehudayoff [7].

Lemma 7.2 (Rational continuous-expander interface)**.** *There are a finite nonempty family of rational increasing partial Möbius maps f_j and a real $\varepsilon > 0$ such that each map has a nonempty open interval domain with rational endpoints, takes that domain into $[0,1]$, and*

$$(1+\varepsilon)|S| \le \left| \bigcup_j f_j(S \cap \operatorname{dom} f_j) \right|$$

for every measurable $S \subseteq [0,1]$ with $|S| \le 1/2$. Each map has rational coefficients, positive determinant and positive denominator at both endpoints of its defining closed interval.

Lean: `TransposeFollowup.UnconditionalPaidPadding`
`TransposeFollowup.exists_rational_continuous_expander`

Lean: `TransposeFollowup.RationalMobius`
`TransposeFollowup.RationalMobius`

Outline of the formal proof. The rational free family and a sufficiently fine fixed grid give a restricted L^2 gap for functions supported on $[0,1]$ whose integral on each grid cell is zero. If a measurable set expanded too little under the family and the auxiliary translations, its cell-centered, normalized indicator would have large correlation with every generator and inverse. Averaging contradicts the restricted gap. Clipping the augmented rational family supplies the open domains and removes endpoints of measure zero. The same rational coefficients pass through the gap and expansion interfaces; no separate unverified approximation of an expander is substituted.

The lemma and its analytic dependencies are proved in the checked library. The pinned OpenAI dependency is a formally proved scalar projection result; it is separate from the OpenAI multiplication source cited in the earlier paper.

A short dependency route. The following links locate the construction without requiring the reader to follow the compiler's complete internal file graph.

Obligation	Principal interface
Rational gap and expansion	`UnconditionalPaidPadding`
All-size finite expansion	`AllSizeOverlapExpansion`, `ContinuousNativePadding`
Generated actual trajectories	`NativeTimedWalk`, `PaidPaddingLimits`
Cleanup and blank payload entry	`GeneratorCleanupCost`, `PaidPaddingLimits`
Full-horizon cut inequality	`PaidPayload.cut_after_prefix` in `PaidPaddingLimits`
Original limiting conclusion	`NativeRunLimits.of_cut`

7.4 What the construction establishes

The earlier paper's expanding-motion proposal [9, Section 13, "The construction side"] requires rulers that drive an encoder along an expanding relation graph of its data, at every recursive level.

Theorem 7.1 supplies a data-free version: a uniform machine generates and executes expanding excursions in linear time, with every setup and execution step charged. The excursions run once before the payload and carry no matrix data. Coupling this motion to the encoder and decoder at successive recursive levels remains open.

For the impossibility program, apply the theorem to a machine that halts immediately with empty output. The wrapper is a fixed linear-time machine that outputs the empty word on every positive-dimensional matrix input, yet satisfies the zero-slack cut inequality and the full M^* conclusions. Thus these geometric conditions can hold without performing any transpose. A transposition lower bound must also use the output demand or another property not supplied by the wrapper. This conclusion concerns the stated cut inequality, including its coarse memory-boundary charge.

The theorem uses $O(P)$ schedule generation. It does not assert an $o(P)$ generator, a particular weak-limit return measure, or a nonhyperfinite return relation.

8 What remains to decide

The results leave two questions: which coded updates can recover the required outputs, and whether those updates can be executed in linear time on tapes. The local-coding theorems exclude specific information-access patterns. The motion theorems exclude specific execution patterns. The padding construction supplies the cut condition, and for linear-time payloads the full limiting conditions, without requiring a transpose-specific computation.

An impossibility target. The earlier paper proves that (T) gives causal complete-exchange codes on valid tape diagrams, at the prescribed block capacities, on input sets with cardinality deficit at most Cd for a fixed C. The diagrams append their output in the last P steps. It therefore suffices to show that, for every fixed choice of resource constants and C, arbitrarily large dimensions admit no such diagram and code. This requires less than the full robust Tape Coding Conjecture, which excludes a larger class of input sets at every sufficiently large size.

Imported Lean: `Transpose.Tape.Statements.Coding`
`Transpose.Tape.codes_coentLinear_of_linearTimeTransposable`
`Transpose.Tape.not_linearTimeTransposable_of_coentLinear_end_io`

Applying our local-coding bounds to this target requires a structural argument: one must prove that the extracted tape codes satisfy the relevant locality or factorization hypotheses. An evolving coded bank is not an original row, and an arbitrary execution need not factor into the two stages of our helper theorem. Improving constants in those bounds does not supply this missing argument.

A lower-bound invariant to test. A further lower bound must use more than the aggregate cut and M^* conditions supplied by the padding wrapper. One candidate is an invariant that tracks both causal availability of coded information and its use in recovering specified outputs. A useful test is whether adjoining irrelevant padding leaves the invariant unchanged. The next tests are whether it survives the correlated and nonlinear examples allowed by our local theorems, and whether its accumulated cost is superlinear in the full tape model. This is a proposed direction; we have not proved such an invariant exists.

A construction target. The next constructive step is an update and decoding identity with a proved saving. Its state invariant must specify what is stored, which data each update can access, and how the next outputs are recovered. The cost must include helper and layout generation,

encoding, decoding, head movement, finite control and raw output. A saving in an abstract coding model alone does not establish (T).

The first paper's code-gap theorem requires an $\Omega(\log d)$ advantage over routing on the relevant tape networks. A code with this gain would still need a uniform finite-state implementation with linear total cost.

Imported Lean: `Transpose.Tape.Statements.Coding`
`Transpose.Tape.code_gap`

Several possibilities remain outside our exclusions: high-treewidth queries on original rows, more general broadcasts than fixed coordinate XORs, more than two nonlinear stages, and recursive updates that fail the stated affine-regularity hypotheses. None is thereby shown feasible. The padding construction supplies motion, not an encoder or decoder for complete exchange.

Evaluating a candidate. State its information access, helper costs, correctness set and actual tape costs first. Then evaluate the applicable quantitative bounds. Discard the candidate when a bound contradicts its claimed cost; membership in a local model alone is not enough. If no exclusion applies, identify the exact hypothesis it escapes and test the remaining computation directly.

A Building and checking the artifacts

A.1 Shared sources and trust boundary

The accompanying project uses Lean 4 and Mathlib at version 4.34.1 [12, 29, 30]. The mathematical source is in `lean/TransposeFollowup/`, with the reused native compiler in `lean/FixedTapeCompiler/`. The local package `lean/upstream/` points directly to the sibling first paper's sources. Its build outputs are kept in the adapter; no definitions or proofs from that paper are copied or modified. The original frozen statement, checker, and source inventory and digests are checked by `verification/check_upstream.py`. Its record separately identifies a subsequent editorial update to seven first-paper artifacts; the first paper's Lean proofs are unchanged.

The external OpenAI source is pinned to commit `1581d4cbdd69a0faf34b2089614fe2a814316a6a`. An exact 204-module import closure, with its original namespace and Apache-2.0 license, is installed by `verification/fetch_openai_metric.py`. The manifest `verification/openai-metric-source.json` records import edges and every source digest. The bibliography and attribution ledger identify the imported scalar projection and related combinatorial estimates. The adapter preserves these sources and uses its own package configuration.

The proofs use only the standard axioms `propext`, `Classical.choice` and `Quot.sound`. The manuscript-specific audit runs `#print axioms` on all explicitly cited formal declarations, including definitions and imported endpoints. It checks that every requested declaration has an audit result and that no other axiom occurs. The formal source guard additionally checks preservation of the audited mathematical sources and trust settings; the review records separately identify two comment-only corrections in the padding module. This is an ordinary build and transitive axiom audit, not a fresh empty-environment replay of every imported proof term.

A.2 Reproduction

Keep the two paper folders adjacent in the repository. With the pinned Lean toolchain available through `elan`, run the following commands from the follow-up folder. The external source installer can use `--source-root PATH` for an offline source checkout; its digest checks are the same.

```
python3 verification/fetch_openai_metric.py
python3 verification/check_upstream.py
cd lean
lake exe cache get
cd ..
python3 verification/manuscript/verify.py
python3 verification/manuscript/build_paper.py
```

The verification command generates the declaration inventory, runs `lake build`, builds all cited modules, and runs the manuscript axiom audit. For the broader support audit, run

```
python3 verification/run_axiom_audit.py support
```

This also builds every import before invoking Lean. A bare library build may omit first-paper modules used only by the audit entry point. The verification command also checks the protected sources and retains the commands, logs and source hashes. A TeX distribution with `latexmk`, BibTeX and the packages named in `paper/main.tex` builds the manuscript. The final PDF is `paper/main.pdf`; intermediate TeX files remain in `paper/build/`.

A.3 What the verification establishes

The declaration inventory covers every formal annotation in the manuscript, including numbered statements, definitions and cited background endpoints. Generation checks module existence; the Lean audit checks declaration existence and transitive axioms. A separate semantic review compares the hypotheses, conclusions and quantifier order of the English statements with the formal source. The machine-readable contracts and review resolutions are retained in `verification/manuscript/`.

The earlier library audit checked 6,313 declarations across the six theorem families; the manuscript audit checks those cited here. Source reviews were performed by AI agents and are not external human peer review.

Compilation is followed by checks for missing references, citations and formatting overflow, PDF text inspection and rendered-page review. The source ledger separately records imported theorems, borrowed arguments and literature comparisons. Neither rendering checks nor kernel checks establish novelty.

B Declaration and dependency index

The primary endpoints of every numbered result are listed below. The complete inventory, including definitions and imported results cited in the text, is generated as `verification/manuscript/declarations.json`. It records source modules, manuscript locations and source hashes. All names in that inventory are checked by `lean/PaperAxioms.lean`.

Definition 2.1. The unchanged decision target

`Transpose.Statement`

`Transpose.entryBits`

`Transpose.inputLength`

`Transpose.encode`

`Transpose.transpose`

`Transpose.TransposesWithin`

`Transpose.LinearTimeTransposable`

Definition 4.1. Public original-row strategy
`TransposeFollowup.ProtocolTruncation`

`TransposeFollowup.PublicStrategy`

`TransposeFollowup.PublicStrategy.Executes`

Theorem 4.2. Adaptive separator bound
`TransposeFollowup.GraphSeparatorBound`

`TransposeFollowup.graph_expected_separator_bound_of_distribution`

`TransposeFollowup.graph_expected_separator_bound`

Corollary 4.3. Worst-case cost and attained helper range
`TransposeFollowup.GraphSeparatorBound`

`TransposeFollowup.graph_worst_case_separator_bound`

Theorem 4.4. Separate sharpness of the two information charges
`TransposeFollowup.SeparatorSharpness`

`TransposeFollowup.SeparatorSharpness.deficit_coefficient_at_least_two`

`TransposeFollowup.SeparatorSharpness.helper_coefficient_at_least_two`

Lemma 4.5. Integral fragmentation
`TransposeFollowup.Treewidth`

`TransposeFollowup.treewidth_fragmentation`

`TransposeFollowup.treewidth_separator_loss`

Corollary 4.6. Savings force large average treewidth
`TransposeFollowup.AdaptiveTreewidth`

`TransposeFollowup.average_treewidth_lower_bound_of_distribution`

`TransposeFollowup.average_treewidth_lower_bound`

Definition 4.7. Recoded fixed-XOR code
`TransposeFollowup.RecodedXor`

`TransposeFollowup.RecodedXorCode`

`TransposeFollowup.RecodedXorCode.Correct`

`TransposeFollowup.RecodedXorCode.LocalTo`

Theorem 4.8. Distance bound after arbitrary row recoding
`TransposeFollowup.XorComponentCount`

`TransposeFollowup.RecodedXorCode.broadcast_lower_bound_scaled`

`TransposeFollowup.RecodedXorCode.broadcast_lower_bound`

Corollary 4.9. Bounded degree
`TransposeFollowup.XorDegreeBound`

`TransposeFollowup.RecodedXorCode.fixed_degree_broadcast_lower_bound`

Corollary 4.10. Paths, cycles, and their embedded subgraphs
`TransposeFollowup.XorPathCycleBound`

`TransposeFollowup.RecodedXorCode.linear_ball_broadcast_lower_bound`

`TransposeFollowup.RecodedXorCode.cycle_embedding_broadcast_lower_bound`

`TransposeFollowup.RecodedXorCode.subcycle_broadcast_lower_bound`

TransposeFollowup.RecodedXorCode.cycle_broadcast_lower_bound

TransposeFollowup.RecodedXorCode.path_broadcast_lower_bound

Definition 4.11. Triangular and causal row maps

TransposeFollowup.TriangularRows

TransposeFollowup.TriangularRows

TransposeFollowup.TriangularRows.apply

TransposeFollowup.CausalRows

TransposeFollowup.CausalRows.apply

Definition 4.12. Two-stage branch and helper selection

TransposeFollowup.TwoStageRows

TransposeFollowup.TwoStageRows

TransposeFollowup.TwoStageRows.apply

Lemma 4.13. Block injectivity

TransposeFollowup.TriangularRows

TransposeFollowup.TriangularRows.block_injective

TransposeFollowup.TriangularRows.equiv

TransposeFollowup.blockInjective_rowwise

TransposeFollowup.blockInjective_inverse

Lemma 4.14. A record for one branch

TransposeFollowup.TwoStageRows

TransposeFollowup.TwoStageRows.correct_family_card

Theorem 4.15. Exact helper-or-deficit toll

TransposeFollowup.TwoStageHelper

TransposeFollowup.TwoStageHelper.correct_family_count_toll

TransposeFollowup.TwoStageHelper.quarter_count_toll

TransposeFollowup.TwoStageHelper.quarter_helper_bound_of_density

Theorem 4.16. Actual helper entropy under arbitrary input laws

TransposeFollowup.TwoStageHelperEntropy

TransposeFollowup.TwoStageHelper.helper_entropy_deficit_bound_of_distribution

TransposeFollowup.TwoStageHelper.quarter_helper_entropy_deficit_bound_of_distribution

TransposeFollowup.TwoStageHelper.helper_entropy_deficit_bound

TransposeFollowup.TwoStageHelper.quarter_helper_entropy_deficit_bound

Definition 5.1. The restricted star condition

TransposeFollowup.FiniteAffineTimeBound

TransposeFollowup.AffineMotion.SatisfiesStar

Theorem 5.2. Quantitative affine-time obstruction

TransposeFollowup.OpenAffineTimeBound

TransposeFollowup.AffineMotion.star_half_le_locally_affine_omission

TransposeFollowup.AffineMotion.star_half_le_affine_component_omission

TransposeFollowup.AffineMotion.star_common_affine_omission

Theorem 5.3. Physical units and native limiting paths
`TransposeFollowup.NativeAffineMotion`

`TransposeFollowup.AffineMotion.popular_affine_time_nonuniform`

`TransposeFollowup.AffineMotion.popular_affine_time_robust`

`TransposeFollowup.AffineMotion.AffineTimeRequired.physical_omission`

Definition 5.4. Bounded affine interval cover
`TransposeFollowup.BoundedAffineCovers`

`TransposeFollowup.AffineMotion.HasAffineIntervalCover`

Lemma 5.5. Pointwise closure
`TransposeFollowup.BoundedAffineCovers`

`TransposeFollowup.AffineMotion.local_affine_ae_of_eventual_bounded_covers`

Theorem 5.6. Pointwise approximation obstruction
`TransposeFollowup.PhysicalAffineApproximationObstruction`

`TransposeFollowup.AffineMotion.no_subsequential_physical_pointwise_affine_fast_approxi mation`

Theorem 6.1. Bounded-word palette exclusions
`TransposeFollowup.MatrixPaletteObstructions`

`TransposeFollowup.IntegerMatrixPalette.no_high_entropy_transpose`

`TransposeFollowup.TernaryMatrixPalette.no_high_entropy_transpose`

Definition 6.2. Contracting stage clock
`TransposeFollowup.ContractingStageClock`

`TransposeFollowup.RegularMotion.ContractingStageClock`

Lemma 6.3. Charged depth and exceptional work
`TransposeFollowup.StageClockAccounting`

`TransposeFollowup.RegularMotion.stage_clock_deep_work_bound`

`TransposeFollowup.RegularMotion.stage_clock_total_time_bound`

Definition 6.4. Regular sequences of stage occurrences
`TransposeFollowup.ActualStageRegularity`

`TransposeFollowup.RegularMotion.HasRegularStageSubsequences`

Theorem 6.5. Exceptional-time toll for contracting recursion
`TransposeFollowup.NativeExceptionalRecursion`

`TransposeFollowup.RegularMotion.native_flagged_recursion_budget`

`TransposeFollowup.RegularMotion.native_fast_flagged_recursion_budget`

`TransposeFollowup.RegularMotion.uniform_flagged_recursion_budget`

Corollary 6.6. Per-node exceptional budget
`TransposeFollowup.NativeExceptionalRecursion`

`TransposeFollowup.RegularMotion.HasNodeBudgetRecursiveExecutions`

`TransposeFollowup.RegularMotion.native_node_budget_recursion_bound`

Theorem 7.1. Universal fully paid padding
`TransposeFollowup.UnconditionalPaidPadding`

`TransposeFollowup.PaidPadding.exists_uniform_paid_padding`

Lemma 7.2. Rational continuous-expander interface
`TransposeFollowup.UnconditionalPaidPadding`

`TransposeFollowup.exists_rational_continuous_expander`

Dependency structure. The original model and M^* theorem feed the native motion and approximation obstructions, which feed the scan-palette and recursion exclusions. The finite original-row, coordinate-XOR and two-stage interfaces support the three local-coding families independently of the analytic padding chain. For padding, rational expansion feeds finite overlap graphs and their paid excursion implementation; cleanup and payload composition preserve the computation, and the original cut-to-limit theorem supplies the full M^* conclusion. These are logical dependencies; the manuscript order is chosen for exposition rather than to reproduce the import graph.

References

[1] Peyman Afshani, Casper Benjamin Freksen, Lior Kamma, and Kasper Green Larsen. Lower bounds for multiplication via network coding. In *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*, volume 132 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 10:1–10:12, 2019. doi:10.4230/LIPIcs.ICALP.2019.10.

[2] Noga Alon, Ankur Moitra, and Benny Sudakov. Nearly complete graphs decomposable into large induced matchings and their applications. *Journal of the European Mathematical Society*, 15(5):1575–1596, 2013. URL: https://ems.press/journals/jems/articles/11869, doi:10.4171/JEMS/398.

[3] Ziv Bar-Yossef, Yitzhak Birk, T. S. Jayram, and Tomer Kol. Index coding with side information. *IEEE Transactions on Information Theory*, 57(3):1479–1494, 2011. doi:10.1109/TIT.2010.2103753.

[4] Anna Blasiak, Robert Kleinberg, and Eyal Lubetzky. Index coding via linear programming, 2011. Version 2, 13 July 2011. URL: https://arxiv.org/abs/1004.1379v2, arXiv:1004.1379.

[5] Jean Bourgain. On the Erdős–Volkmann and Katz–Tao ring conjectures. *Geometric and Functional Analysis*, 13:334–365, 2003. doi:10.1007/s000390300008.

[6] Jean Bourgain. The discretized sum-product and projection theorems. *Journal d'Analyse Mathématique*, 112:193–236, 2010. doi:10.1007/s11854-010-0028-x.

[7] Jean Bourgain and Amir Yehudayoff. Expansion in $\mathrm{SL}_2(\mathbb{R})$ and monotone expanders. *Geometric and Functional Analysis*, 23(1):1–41, 2013. doi:10.1007/s00039-012-0200-9.

[8] Mark Braverman and Zhongtian He. The multiple unicast conjecture is false. arXiv:2610.10108, 2026. URL: https://arxiv.org/abs/2610.10108.

[9] Hans Brende and Claude Opus 5.5 (Anthropic). Matrix transposition and the tape coding conjecture. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 5f99ef1a503f5d56585a3862b2778193b6d358f4. URL: https://github.com/HansBrende/papers/tree/5f99ef1a503f5d56585a3862b2778193b6d358f4/Matrix-Transposition-and-the-Tape-Coding-Conjecture.

[10] Hari Hara Suthan Chittoor, Prasad Krishnan, K V Sushena Sree, and Bhavana MVN. Subexponential and linear subpacketization coded caching via projective geometry, 2021. First preprint 2020; checked v2, November 28, 2021, particularly Scheme C and Corollary 2. URL: https://arxiv.org/abs/2001.00399, arXiv:2001.00399, doi:10.1109/TIT.2021.3095471.

[11] Fan R. K. Chung, Ronald L. Graham, Peter Frankl, and James B. Shearer. Some intersection theorems for ordered sets and graphs. *Journal of Combinatorial Theory, Series A*, 43(1):23–37, 1986. doi:10.1016/0097-3165(86)90019-1.

[12] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction—CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi:10.1007/978-3-030-79876-5_37.

[13] Martin Dietzfelbinger and Wolfgang Maass. The complexity of matrix transposition on one-tape off-line Turing machines with output tape. *Theoretical Computer Science*, 108(2):271–290, 1993. doi:10.1016/0304-3975(93)90194-X.

[14] Martin Dietzfelbinger, Wolfgang Maass, and Georg Schnitger. The complexity of matrix transposition on one-tape off-line Turing machines. *Theoretical Computer Science*, 82(1):113–129, 1991. doi:10.1016/0304-3975(91)90175-2.

[15] Zeev Dvir and Avi Wigderson. Monotone expanders: Constructions and applications. *Theory of Computing*, 6(12):291–308, 2010. URL: https://theoryofcomputing.org/articles/v006a012/, doi:10.4086/toc.2010.v006a012.

[16] Erling Følner. On groups with full Banach mean value. *Mathematica Scandinavica*, 3:243–254, 1955. doi:10.7146/math.scand.a-10442.

[17] David Harvey and Joris van der Hoeven. Integer multiplication is at least as hard as matrix transposition. arXiv:2503.22848, 2025. URL: https://arxiv.org/abs/2503.22848.

[18] Maria Klawe. Limitations on explicit constructions of expanding graphs. *SIAM Journal on Computing*, 13(1):156–166, 1984. doi:10.1137/0213011.

[19] Min Li, Lawrence Ong, and Sarah J. Johnson. Cooperative multi-sender index coding. *IEEE Transactions on Information Theory*, 65(3):1725–1739, 2019. doi:10.1109/TIT.2018.2869161.

[20] Min Li, Lawrence Ong, and Sarah J. Johnson. Multi-sender index coding for collaborative broadcasting: a rank-minimization approach. *IEEE Transactions on Communications*, 67(2):1452–1466, 2019. doi:10.1109/TCOMM.2018.2877392.

[21] Eyal Lubetzky and Uri Stav. Non-linear index coding outperforming the linear optimum, 2008. URL: https://arxiv.org/abs/0806.1919, arXiv:0806.1919.

[22] Alexander Lubotzky and Benjamin Weiss. Groups and expanders. In *Expanding Graphs*, volume 10 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 95–109. American Mathematical Society, 1993. doi:10.1090/dimacs/010/08.

[23] Kunal Mittal. *Parallel Repetition of Multiplayer Games: New Bounds and Applications*. PhD thesis, Princeton University, September 2025. Section 10.8.2, Proposition 10.8.3 and Lemma 10.8.5, pp. 158–160; the proof is attributed to Mark Braverman and Katherine Forbes, personal communication, 2022. URL: https://dataspace.princeton.edu/handle/88435/dsp01xd07gx16m.

[24] Kunal Mittal and Ran Raz. Block rigidity: Strong multiplayer parallel repetition implies super-linear lower bounds for turing machines. In *12th Innovations in Theoretical Computer Science Conference (ITCS 2021)*, volume 185 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 71:1–71:15, 2021. Conjecture 36; its transpose prediction assertion is false. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITCS.2021.71, doi:10.4230/LIPIcs.ITCS.2021.71.

[25] Lawrence Ong, Chin Keong Ho, and Fabian Lim. The single-uniprior index-coding problem: The single-sender case and the multi-sender extension. *IEEE Transactions on Information Theory*, 62(6):3165–3182, 2016. URL: https://arxiv.org/abs/1412.1520v2, arXiv:1412.1520, doi:10.1109/TIT.2016.2555950.

[26] OpenAI. ScalarWeakRadiusProjection: a scalar projection theorem in the OAI formal library. Lean source, repository openai/math, 2026. Revision 1581d4cbdd69a0faf34b2089614fe2a814316a6a; headline declaration `scalar_weak_radius_projection` in namespace `OAI.Falconer.Regularization`, with its pinned support-module closure. This entry cites formal sources, not an associated article. URL: https://github.com/openai/math/blob/1581d4cbdd69a0faf34b2089614fe2a814316a6a/lean/OAI/MeasureTheory/Falconer/Geometry/ScalarWeakRadiusProjection.lean.

[27] Parastoo Sadeghi, Fatemeh Arbabjolfaei, and Young-Han Kim. Distributed index coding. In *2016 IEEE Information Theory Workshop (ITW)*, pages 330–334, 2016. arXiv:1604.03204, doi:10.1109/ITW.2016.7606850.

[28] Karthikeyan Shanmugam, Antonia M. Tulino, and Alexandros G. Dimakis. Coded caching with linear subpacketization is possible using Ruzsa–Szemerédi graphs, 2017. Checked v1; the induced-matching application uses density followed by minimum-degree pruning. URL: https://arxiv.org/abs/1701.07115, arXiv:1701.07115.

[29] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. ACM, 2020. arXiv:1910.09336, doi:10.1145/3372885.3373824.

[30] The mathlib Community. Mathlib4. Formal mathematics library, Lean 4 version, 2026. Pinned revision d13f23b723b8a846827a245b89c10fc7d3f11612, release v4.34.1. URL: https://github.com/leanprover-community/mathlib4/tree/d13f23b723b8a846827a245b89c10fc7d3f11612.

[31] Qifa Yan, Minquan Cheng, Xiaohu Tang, and Qingchun Chen. On the placement delivery array design in centralized coded caching scheme. *IEEE Transactions on Information Theory*, 63(9):5821–5833, 2017. First preprint 2015; checked arXiv v3, Definition 1 and Theorem 4. URL: https://arxiv.org/abs/1510.05064, arXiv:1510.05064, doi:10.1109/TIT.2017.2725272.

[32] Xindan Zhang, Baochun Li, and Zongpeng Li. Network coding can beat routing in undirected multiple-unicast networks. arXiv:2610.09367, 2026. URL: https://arxiv.org/abs/2610.09367.